TRS-80 Level II BASIC

written in Starlark

```
RADIO SHACK LEVEL II BASIC

READY
>_
```

A complete Level II BASIC interpreter, about 3,000 lines of Starlark.
The host program only supplies the keyboard and the screen.

Source files:

```
basic.star      1289 lines
expr.star        740 lines
lex.star         188 lines
numbers.star     438 lines
screen.star      330 lines
editor.star      204 lines
```

Printed 2026-11-04 from the files in the repository.

Contents

How it works 3

Sample programs 4

basic.star 6

The machine: program storage, statements, control flow, errors, INPUT, the run loop

expr.star 25

Expressions: compiled to reverse Polish notation, then evaluated, with no recursion

lex.star 36

The tokenizer: keywords anywhere in a line, two-character names, numbers

numbers.star 39

Numbers: integer, single and double precision, printing, and the math functions

screen.star 46

The 64 x 16 screen, block graphics and PRINT USING

editor.star 51

The EDIT command as a function of one keystroke

How it works

Starlark has no while loop and no recursion, so the interpreter is one flat loop. run() executes one BASIC statement per turn of a bounded for loop. GOTO, GOSUB and FOR are only changes to the program counter and to two plain lists used as stacks. Expressions are compiled with the shunting-yard algorithm and evaluated with an explicit stack, so nesting depth is a number we choose and can never overflow the host.

The whole machine is one dictionary. The host calls submit(state, line, budget) with each typed line and takes back what was printed. Nothing in the script touches the console, the disk or the clock, and RND is a generator written in the script, so the same program prints the same thing every time. INPUT works by returning "waiting" and being called again with the answer.

What it reproduces from Level II

- Line-numbered programs; RUN, LIST, NEW, CONT, DELETE, CLEAR and EDIT (the key-by-key line editor).
- PRINT with ; , TAB(SPC(@ and USING; LET, IF/THEN/ELSE, FOR/NEXT/STEP, GOTO, GOSUB/RETURN, ON..GOTO/GOSUB, INPUT, DATA/READ/RESTORE.
- DIM, DEF FN, DEFINT/SNG/DBL/STR, RANDOM, CLS, SET/RESET/POINT, PEEK/POKE on video memory, TRON/TROFF, STOP, END, ERROR, ON ERROR GOTO, RESUME.
- Integer, single and double precision numbers (a single is rounded to 24 bits) and strings; the usual functions; two-letter error codes (?SN ERROR IN 100).
- Keywords are found anywhere in a line, so FORI=1TO10 works with no spaces, as on the real machine, and names have two significant characters.
- A FOR loop always runs its body once. True is -1. Numbers print with a sign slot and a trailing space.

Where it differs

- SIN, COS, TAN, ATN, EXP, LOG and ^ are series approximations written in the script, accurate to the six digits Level II printed.
- RND is a seeded generator of our own, so the numbers differ from a real TRS-80's; they are repeatable.
- Disk and cassette, LPRINT, AUTO and RENUM are not implemented. Programs are stored as typed text, not as token bytes, so PEEK of program memory does not match.
- The details of EDIT were written from memory of the manual, not checked against a real machine or a ROM.

Sample programs

Each program is listed, then the output the interpreter produced when it ran it for this document.

hello.bas

```
10 REM HELLO AND A TABLE OF SQUARES
20 PRINT "HELLO FROM TRS-80 LEVEL II BASIC"
30 FOR I=1 TO 5
40 PRINT I, I*I, SQR(I)
50 NEXT I
60 END
```

Output

```
 HELLO FROM TRS-80 LEVEL II BASIC
  1              1              1
  2              4              1.41421
  3              9              1.73205
  4              16             2
  5              25             2.23607
 READY
 >
```

fib.bas

```
10 REM FIBONACCI WITH DEF FN AND STRINGS
20 DEF FNA(X)=X*X+1
30 A=0:B=1
40 FOR I=1 TO 12
50 PRINT A;
60 C=A+B:A=B:B=C
70 NEXT I
80 PRINT
90 PRINT FNA(3);FNA(4)
100 A$="TRS-80":B$=LEFT$(A$,3)+MID$(A$,4,1)+RIGHT$(A$,2)
110 PRINT B$;LEN(B$);ASC("A");CHR$(66)
120 PRINT STR$(42);"!";VAL("3.5E2")
```

Output

```
  0  1  1  2  3  5  8  13  21  34  55  89
 10  17
TRS-80 6  65 B
 42! 350
READY
>
```

primes.bas

```
10 REM SIEVE OF ERATOSTHENES
20 N=100
30 DIM P(N)
40 FOR I=2 TO N:P(I)=1:NEXT I
50 FOR I=2 TO N
60 IF P(I)=0 THEN 100
70 PRINT I;
80 IF I*I>N THEN 100
90 FOR J=I*I TO N STEP I:P(J)=0:NEXT J
100 NEXT I
110 PRINT
```

Output

```
 2  3  5  7  11  13  17  19  23  29  31  37  41  43  47  53  59  61  67  71  73  79  83  89  97
READY
>
```

sine.bas

```
10 REM FOUR PERIODS OF A SINE WAVE, 60 COLUMNS BY 15 ROWS
20 PI=3.14159265
30 DIM Y(59)
40 FOR X=0 TO 59
50 Y(X)=INT(7-7*SIN(X*2*PI/15)+.5)
60 NEXT X
70 FOR R=0 TO 14
80 FOR X=0 TO 59
90 IF Y(X)=R THEN PRINT "*";:GOTO 130
100 IF R=7 THEN PRINT "-";:GOTO 130
110 PRINT " ";
130 NEXT X
140 PRINT
150 NEXT R
160 PRINT "4 PERIODS OF SIN(X), 15 COLUMNS EACH"
```

Output

```
    **            **            **            **
      *            *            *            *
   *            *            *            *
       *            *            *            *
 *            *            *            *

       *            *            *            *
*-------------*-------------*-------------*-------------
        *            *            *            *

             *            *            *            *
          *            *            *            *
              *            *            *            *
            *            *            *            *
             **            **            **            **
4 PERIODS OF SIN(X), 15 COLUMNS EACH
READY
>
```

guess.bas

```
10 REM GUESS THE NUMBER
20 RANDOM
30 N=INT(RND(100))
40 PRINT "I AM THINKING OF A NUMBER FROM 1 TO 100"
50 T=0
60 INPUT "YOUR GUESS";G
70 T=T+1
80 IF G<N THEN PRINT "TOO LOW":GOTO 60
90 IF G>N THEN PRINT "TOO HIGH":GOTO 60
100 PRINT "YOU GOT IT IN";T;"GUESSES"
```

Output

```
I AM THINKING OF A NUMBER FROM 1 TO 100
YOUR GUESS? 50
TOO HIGH
YOUR GUESS? 25
YOU GOT IT IN 2 GUESSES
READY
>
```

The source

Starlark, in the order a reader would want it: the machine first, then what it uses. Colours are from the library's own lexer.

basic.star

The machine: program storage, statements, control flow, errors, INPUT, the run loop

```
# TRS-80 Level II BASIC, written in Starlark.
#
# The whole machine is one dict, the state `st`. The host (C# or a test) calls new_state(), then feeds it lines with
# submit(st, text, budget) and keeps calling run(st, budget) until it returns "done" or "waiting" (for an INPUT). It
# collects what was printed with take_output(st). Nothing here touches the console, the clock or the disk, and RND is a
# generator written in this program, so a BASIC program prints exactly the same thing every time it is run.
#
# Starlark has no `while` and no recursion, so the interpreter is one flat loop: run() executes one BASIC statement per turn
# of a bounded `for`, and BASIC's own GOTO, GOSUB and FOR are just changes to the program counter (st["line"], st["pos"])
# and to two plain lists used as stacks. Expressions are compiled and evaluated without recursion too (see expr.star).
#
# What is reproduced from Level II: line-numbered programs; the direct commands RUN, LIST, NEW, CONT, DELETE, CLEAR; PRINT
# with ; , TAB( SPC( @ and USING; LET, IF/THEN/ELSE, FOR/NEXT/STEP, GOTO, GOSUB/RETURN, ON..GOTO/GOSUB, INPUT, DATA/READ/
# RESTORE, DIM, DEF FN, DEFINT/SNG/DBL/STR, RANDOM, CLS, SET/RESET/POINT, PEEK/POKE on video memory, TRON/TROFF, STOP, END,
# ERROR, ON ERROR GOTO, RESUME; integer, single and double precision numbers and strings; the usual functions; and the
# two letter error codes ("?SN ERROR IN 100"). A FOR loop runs its body once even when the start is already past the limit,
# as Level II did.

load("numbers.star", "INT_MAX", "INT_MIN", "floor_int", "to_float", "kind_of", "span_digits", "parse_number_text", "print_text")
load("lex.star", "tokenize")
load("editor.star", "new_edit", "edit_step", "edit_text", "edit_head")
load("expr.star", "eval_at", "compile_expr", "var_key", "key_kind", "zero_of", "coerce", "locate", "new_array", "binop")
load("screen.star", "new_screen", "put_text", "newline", "clear_screen", "set_cursor", "set_pixel", "screen_lines",
     "format_using", "COLUMNS", "CELLS", "VIDEO_BASE")

# The two-letter error codes in the order of the BASIC `ERROR n` statement: ERROR 1 is "NF", ERROR 2 is "SN", and so on. `ERR` reports twice the
# position here, as Level II did (its codes were even numbers). A statement that fails returns one of these strings; handle_error prints it.
ERRORS = ["NF", "SN", "RG", "OD", "FC", "OV", "OM", "UL", "BS", "DD", "/0", "ID", "TM", "OS", "LS", "ST", "CN", "NR", "RW", "UE",
          "MO", "FD", "L3", "UF"]

# Long names, for hosts that want to explain a code. The machine itself printed only the two letters.
ERROR_NAMES = {
    "NF": "NEXT without FOR", "SN": "Syntax error", "RG": "RETURN without GOSUB", "OD": "Out of DATA",
    "FC": "Illegal function call", "OV": "Overflow", "OM": "Out of memory", "UL": "Undefined line number",
    "BS": "Subscript out of range", "DD": "Redimensioned array", "/0": "Division by zero", "ID": "Illegal direct",
    "TM": "Type mismatch", "OS": "Out of string space", "LS": "String too long", "ST": "String formula too complex",
    "CN": "Can't continue", "NR": "No RESUME", "RW": "RESUME without error", "UE": "Unprintable error",
    "MO": "Missing operand", "FD": "Bad file data", "L3": "Disk BASIC only", "UF": "Undefined user function",
}

LETTERS = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

# ------------------------------------------------------------------ the machine
```

```
44
45  # Build a fresh machine and show the power-on banner and the READY prompt. The machine is this one dict
      `st`; every other function takes it first.
46  # The keys fall into groups: the stored program and its caches, variables and arrays, where execution is
      (line, toks, pos), the two control
47  # stacks (gosub, fors), DATA, the screen, the error handler, and the line editor.
48  def new_state():
49      st = {
50          "program": {},          # line number -> text after the number
51          "order": [],            # line numbers in order
52          "index": {},            # line number -> position in order
53          "dirty": False,         # order/index need rebuilding
54          "tokcache": {},         # line -> (tokens, error)
55          "rpn": {},              # compiled expressions, keyed by line * 4096 + token position
56          "vars": {}, "arrays": {}, "fns": {},
57          "deftype": {c: 1 for c in LETTERS.elems()},
58          "line": 0, "toks": [], "tokerr": "", "pos": 0, "direct": ([], ""),
59          "running": False, "waiting": None, "cont": None,
60          "gosub": [], "fors": [],
61          "data": [], "dptr": 0,
62          "screen": new_screen(), "cur": 0, "col": 0, "out": [], "quiet": False, "echo": True,
63          "mem": {}, "inkey": [], "seed": 12345, "trace": False,
64          "edit": None, "onerr": 0, "inerr": False, "errline": 0, "errcode": 0, "resume": (0, 0),
      "stmt_start": (0, 0),
65          "steps": 0,
66      }
67      put_text(st, "RADIO SHACK LEVEL II BASIC\n\n")
68      ready(st)
69      return st
70
71  # Return everything printed since the last call as one string, and empty the buffer. The host calls this
      after each step to show the user.
72  def take_output(st):
73      text = "".join(st["out"])
74      st["out"] = []
75      return text
76
77  # The 16 lines of the 64-column screen as a list of strings, trailing spaces removed (see screen_lines).
78  def get_screen(st):
79      return screen_lines(st)
80
81  # Print "READY" and the ">" prompt on a fresh line. The end of every command, program run and error.
82  def ready(st):
83      if st["col"] != 0:
84          newline(st)
85      put_text(st, "READY\n>")
86
87  # Just the ">" prompt on a new line, with no READY: shown after a program line is stored and after a bare
      Enter.
88  def prompt_again(st):
89      if st["col"] != 0:
90          newline(st)
91      put_text(st, ">")
92
93  # Leave run mode and show READY. Used when a program ends, runs off its last line, or hits an error.
94  def stop(st):
95      st["running"] = False
96      ready(st)
97
98  # ---------------------------------------------------------------- program storage
99
100 # Rebuild `order` (line numbers sorted) and `index` (line number -> position) if the program changed since
      they were built. Cheap to call often.
101 def ensure_index(st):
102     if st["dirty"]:
103         order = sorted(st["program"].keys())
104         st["order"] = order
105         st["index"] = {order[i]: i for i in range(len(order))}
106         st["dirty"] = False
107
108 # The tokens of a program line, tokenized on first use and cached in `tokcache`. The cache entry is
      dropped whenever the line changes.
109 def get_toks(st, line):
110     hit = st["tokcache"].get(line)
111     if hit == None:
```

```
        hit = tokenize(st["program"][line])
        st["tokcache"][line] = hit
    return hit

# Make `line` the line being executed: load its tokens into `toks`/`tokerr`. Line 0 is direct mode and uses the tokens of the typed command.
def load_line(st, line):
    if line == 0:
        st["toks"] = st["direct"][0]
        st["tokerr"] = st["direct"][1]
    else:
        t = get_toks(st, line)
        st["toks"] = t[0]
        st["tokerr"] = t[1]
    st["line"] = line

# Go to the start of program line `line` (GOTO, GOSUB, RUN, ON...GOTO). Returns "UL" (undefined line) if it does not exist, otherwise "".
# With TRON on, also prints the line number in square brackets.
def jump_to(st, line):
    if line not in st["program"]:
        return "UL"
    load_line(st, line)
    st["pos"] = 0
    if st["trace"]:
        put_text(st, "[%d]" % line)
    return ""

# Fall through to the line after the current one. In direct mode, or after the last line, the run ends instead.
def next_line(st):
    if st["line"] == 0:
        stop(st)
        return ""
    ensure_index(st)
    i = st["index"].get(st["line"])
    if i == None or i + 1 >= len(st["order"]):
        stop(st)
        return ""
    return jump_to(st, st["order"][i + 1])

# Rebuild the DATA list from the program text.
def build_data(st):
    ensure_index(st)
    items = []
    for line in st["order"]:
        toks = get_toks(st, line)[0]
        for k in range(len(toks) - 1):
            if toks[k] == ("k", "DATA") and toks[k + 1][0] == "s":
                for f in split_fields(toks[k + 1][1]):
                    items.append((f[0], f[1], line))   # remember the DATA line, which is where a bad item is reported
    st["data"] = items
    st["dptr"] = 0

# Split the text of an INPUT line or a DATA statement at commas outside quotes. Each field is (text, was quoted).
def split_fields(text):
    fields = []
    cur = ""
    quoted = False
    in_quotes = False
    for ch in text.elems():
        if ch == "\"":
            if not quoted:
                cur = ""    # spaces before the opening quote are not part of the text
            in_quotes = not in_quotes
            quoted = True
        elif ch == "," and not in_quotes:
            fields.append((cur if quoted else cur.strip(), quoted))
            cur = ""
            quoted = False
        elif quoted and not in_quotes:
            pass            # after the closing quote, up to the comma, is ignored
        else:
```

```
            cur += ch
    fields.append((cur if quoted else cur.strip(), quoted))
    return fields

# ---------------------------------------------------------------- errors

# Report error `code`. If ON ERROR GOTO is active, jump to the handler and remember where to RESUME; otherwise print "?XX ERROR IN line" (no
# "IN" in direct mode) and stop. A bad DATA item sets `errat` so the message names its own line.
def handle_error(st, code):
    line = st.pop("errat", st["line"])   # a bad DATA item names its own line
    st["errcode"] = ERRORS.index(code) * 2 if code in ERRORS else 40
    st["errline"] = line
    if st["onerr"] != 0 and not st["inerr"] and line != 0:
        st["resume"] = st["stmt_start"]
        st["inerr"] = True
        if jump_to(st, st["onerr"]) == "":
            return
    if st["col"] != 0:
        newline(st)
    text = "?" + code + " ERROR"
    if line != 0:
        text += " IN %d" % line
    put_text(st, text + "\n")
    st["cont"] = None
    stop(st)

# The BREAK key. Prints "BREAK IN line", remembers where the program was so CONT can carry on, and stops. Does nothing if nothing is running.
def break_program(st):
    if not st["running"]:
        return
    if st["col"] != 0:
        newline(st)
    put_text(st, "BREAK" + ((" IN %d" % st["line"]) if st["line"] != 0 else "") + "\n")
    st["cont"] = (st["line"], st["pos"])
    st["waiting"] = None
    stop(st)

# ---------------------------------------------------------------- statement helpers

# True when the next token ends a statement: the end of the line, a colon, or ELSE.
def at_end(toks, p):
    if p >= len(toks):
        return True
    t = toks[p]
    return (t[0] == "o" and t[1] == ":") or (t[0] == "k" and t[1] == "ELSE")

# Finish a statement: check nothing but the end of the statement is left, then record `p` as where execution continues. Returns "SN" if
# there is junk after the statement, otherwise "". Almost every st_* function returns through this.
def done_stmt(st, toks, p):
    if not at_end(toks, p):
        return "SN"
    st["pos"] = p
    return ""

# True if the token at position p is the operator or punctuation mark `op` (for example "," or "(").
def is_op(toks, p, op):
    return p < len(toks) and toks[p][0] == "o" and toks[p][1] == op

# True if the token at position p is the keyword `word` (for example "THEN").
def is_kw(toks, p, word):
    return p < len(toks) and toks[p][0] == "k" and toks[p][1] == word

# The line number written at toks[p] (a GOTO target, say) as an int, or -1 if that token is not a plain integer.
def line_number(toks, p):
    if p >= len(toks) or toks[p][0] != "n":
        return -1
    v = toks[p][1]
    if type(v) == "tuple":
        return -1
    return floor_int(to_float(v))

```

```
# Evaluate "( e , e , ... )" starting at the open parenthesis. Returns (values, next position, error).
def paren_args(st, toks, p):
    if not is_op(toks, p, "("):
        return ([], p, "SN")
    vals = []
    q = p + 1
    for _ in range(len(toks) + 1):
        r = eval_at(st, toks, q)
        if r[2] != "":
            return ([], q, r[2])
        vals.append(r[0])
        q = r[1]
        if is_op(toks, q, ","):
            q += 1
        elif is_op(toks, q, ")"):
            return (vals, q + 1, "")
        else:
            return ([], q, "SN")
    return ([], q, "SN")

# A variable reference at toks[p]: ("name", subscripts or None). Returns (key, subs, next position, error).
def parse_target(st, toks, p):
    if p >= len(toks) or toks[p][0] != "v":
        return ("", None, p, "SN")
    key = var_key(st, toks[p][1], toks[p][2])
    q = p + 1
    if is_op(toks, q, "("):
        r = paren_args(st, toks, q)
        if r[2] != "":
            return ("", None, q, r[2])
        return (key, r[0], r[1], "")
    return (key, None, q, "")

# Store `v` in the variable `key`, or in the array element `key(subs)`, converting it to the variable's type first. Returns an error code or "":
# "TM" for a type mismatch, "OV" for a number out of range, "LS" for a string over 255 characters, "BS" for a bad subscript.
def assign(st, key, subs, v):
    kind = key_kind(key)
    cv = coerce(kind, v)
    if cv[1] != "":
        return cv[1]
    if kind == 3 and len(cv[0]) > 255:
        return "LS"
    if subs == None:
        st["vars"][key] = cv[0]
        return ""
    loc = locate(st, key, subs)
    if loc[2] != "":
        return loc[2]
    loc[0][1][loc[1]] = cv[0]
    return ""

# A numeric argument as a whole number. Returns (int, "") or (0, "TM") if it is a string. Fractions round down, as BASIC does.
def number_arg(v):
    if kind_of(v) == 3:
        return (0, "TM")
    return (floor_int(to_float(v)), "")

# ---------------------------------------------------------------- statements

# PRINT. Handles `@ position`, USING, and the separators: ";" joins, "," moves to the next 16-column field (or the next line past column 48),
# TAB( and SPC(. A newline is printed at the end unless the statement ends in ; or ,. Numbers print with a leading sign slot and a trailing space.
def st_print(st, toks, pos):
    n = len(toks)
    p = pos
    if is_op(toks, p, "@"):
        r = eval_at(st, toks, p + 1)
        if r[2] != "":
            return r[2]
        loc = number_arg(r[0])
        if loc[1] != "":
```

```
            return loc[1]
        if loc[0] < 0 or loc[0] >= CELLS:
            return "FC"
        set_cursor(st, loc[0])
        p = r[1]
        if is_op(toks, p, ",") or is_op(toks, p, ";"):
            p += 1
    if is_kw(toks, p, "USING"):
        return print_using(st, toks, p + 1)
    newline_needed = True
    for _ in range(n + 1):
        if at_end(toks, p):
            break
        t = toks[p]
        if t[0] == "o" and t[1] == ";":
            newline_needed = False
            p += 1
            continue
        if t[0] == "o" and t[1] == ",":
            newline_needed = False
            col = st["col"]
            if col >= 48:
                put_text(st, "\n")
            else:
                put_text(st, " " * (16 - col % 16))
            p += 1
            continue
        newline_needed = True
        if t[0] == "k" and (t[1] == "TAB(" or t[1] == "SPC("):
            r = eval_at(st, toks, p + 1)
            if r[2] != "":
                return r[2]
            if not is_op(toks, r[1], ")"):
                return "SN"
            c = number_arg(r[0])
            if c[1] != "":
                return c[1]
            if c[0] < 0 or c[0] > 255:
                return "FC"
            if t[1] == "TAB(":
                if c[0] > st["col"]:
                    put_text(st, " " * (c[0] - st["col"]))
            else:
                put_text(st, " " * c[0])
            p = r[1] + 1
            continue
        r = eval_at(st, toks, p)
        if r[2] != "":
            return r[2]
        v = r[0]
        if kind_of(v) == 3:
            put_text(st, v)
        else:
            put_text(st, print_text(v))
        p = r[1]
    if newline_needed:
        put_text(st, "\n")
    return done_stmt(st, toks, p)

# PRINT USING "format"; values: format each value by the template and print the result. A trailing ; or ,
  suppresses the newline.
def print_using(st, toks, pos):
    r = eval_at(st, toks, pos)
    if r[2] != "":
        return r[2]
    if kind_of(r[0]) != 3:
        return "TM"
    fmt = r[0]
    p = r[1]
    if not (is_op(toks, p, ";") or is_op(toks, p, ",")):
        return "SN"
    p += 1
    values = []
    trailing = False
    for _ in range(len(toks) + 1):
```

```
        if at_end(toks, p):
            break
        e = eval_at(st, toks, p)
        if e[2] != "":
            return e[2]
        values.append(e[0])
        p = e[1]
        trailing = False
        if is_op(toks, p, ";") or is_op(toks, p, ","):
            p += 1
            trailing = True
        elif not at_end(toks, p):
            return "SN"
    f = format_using(fmt, values)
    if f[1] != "":
        return f[1]
    put_text(st, f[0])
    if not trailing:
        put_text(st, "\n")
    return done_stmt(st, toks, p)

# Assignment: `LET A=expr`, or just `A=expr`. The target may be an array element.
def st_let(st, toks, pos):
    t = parse_target(st, toks, pos)
    if t[3] != "":
        return t[3]
    p = t[2]
    if not is_op(toks, p, "="):
        return "SN"
    r = eval_at(st, toks, p + 1)
    if r[2] != "":
        return r[2]
    err = assign(st, t[0], t[1], r[0])
    if err != "":
        return err
    return done_stmt(st, toks, r[1])

# Jump to `line`, or "SN" if the number was not a valid line number.
def goto_line(st, line):
    if line < 0:
        return "SN"
    return jump_to(st, line)

# GOTO line.
def st_goto(st, toks, pos):
    return goto_line(st, line_number(toks, pos))

# GOSUB line: push the place to come back to (the line and the position after the number) and jump. At
  most 400 deep ("OM" beyond that).
def st_gosub(st, toks, pos):
    line = line_number(toks, pos)
    if line < 0:
        return "SN"
    if line not in st["program"]:
        return "UL"
    if len(st["gosub"]) >= 400:
        return "OM"
    st["gosub"].append((st["line"], pos + 1))
    return jump_to(st, line)

# RETURN: pop the saved place and carry on after the GOSUB. "RG" if there is no GOSUB to return to.
def st_return(st, toks, pos):
    if len(st["gosub"]) == 0:
        return "RG"
    back = st["gosub"].pop()
    load_line(st, back[0])
    st["pos"] = back[1]
    return ""

# IF cond THEN ... [ELSE ...]. A true condition continues after THEN (a bare number there means GOTO that
  line). A false one skips to the
# matching ELSE, counting nested IFs, or to the next line if there is none.
def st_if(st, toks, pos):
    n = len(toks)
    r = eval_at(st, toks, pos)
```

```
    if r[2] != "":
        return r[2]
    if kind_of(r[0]) == 3:
        return "TM"
    truth = to_float(r[0]) != 0.0
    p = r[1]
    if is_kw(toks, p, "THEN"):
        p += 1
    elif not is_kw(toks, p, "GOTO"):
        return "SN"
    if not truth:
        depth = 0
        q = p
        found = -1
        for _ in range(n + 1):
            if q >= n:
                break
            if toks[q] == ("k", "IF"):
                depth += 1
            elif toks[q] == ("k", "ELSE"):
                if depth == 0:
                    found = q
                    break
                depth -= 1
            q += 1
        if found < 0:
            return next_line(st)
        p = found + 1
    if p < n and toks[p][0] == "n":
        return goto_line(st, line_number(toks, p))
    st["pos"] = p
    return ""

# Reached by running into ELSE after a finished THEN part: the rest of the line belongs to the false branch, so skip it.
def st_else(st, toks, pos):
    # Reached only by running into the ELSE of an IF whose THEN part has just finished: skip the rest of the line.
    return next_line(st)

# FOR v = start TO limit [STEP s]. Sets the variable and pushes a loop frame (variable, limit, step, line, position). Starting a FOR on a
# variable that already has a frame replaces that frame, which also discards any loops nested inside it. At most 100 loops ("OM").
def st_for(st, toks, pos):
    t = parse_target(st, toks, pos)
    if t[3] != "":
        return t[3]
    if t[1] != None:
        return "SN"
    key = t[0]
    if key_kind(key) == 3:
        return "TM"
    p = t[2]
    if not is_op(toks, p, "="):
        return "SN"
    r1 = eval_at(st, toks, p + 1)
    if r1[2] != "":
        return r1[2]
    if not is_kw(toks, r1[1], "TO"):
        return "SN"
    r2 = eval_at(st, toks, r1[1] + 1)
    if r2[2] != "":
        return r2[2]
    step = 1
    p = r2[1]
    if is_kw(toks, p, "STEP"):
        r3 = eval_at(st, toks, p + 1)
        if r3[2] != "":
            return r3[2]
        step = r3[0]
        p = r3[1]
    if kind_of(r1[0]) == 3 or kind_of(r2[0]) == 3 or kind_of(step) == 3:
        return "TM"
    err = assign(st, key, None, r1[0])
```

```
    if err != "":
        return err
    if not at_end(toks, p):
        return "SN"
    frames = [f for f in st["fors"] if f[0] != key]
    if len(frames) >= 100:
        return "OM"
    frames.append((key, r2[0], step, st["line"], p))
    st["fors"] = frames
    st["pos"] = p
    return ""

# NEXT [v[,v...]]. Adds the step to the loop variable and jumps back to just after the FOR if the loop should go on, otherwise pops the frame.
# The test is made here, at NEXT, so a loop body always runs once, as on Level II. "NF" if there is no matching FOR.
def st_next(st, toks, pos):
    n = len(toks)
    p = pos
    keys = []
    for _ in range(n + 1):
        if p < n and toks[p][0] == "v":
            keys.append(var_key(st, toks[p][1], toks[p][2]))
            p += 1
            if is_op(toks, p, ","):
                p += 1
                continue
        break
    if len(keys) == 0:
        keys = [""]
    for key in keys:
        fors = st["fors"]
        f = -1
        if key == "":
            f = len(fors) - 1
        else:
            for k in range(len(fors)):
                if fors[k][0] == key:
                    f = k
        if f < 0:
            return "NF"
        frame = fors[f]
        st["fors"] = fors[:f + 1]
        cur = st["vars"].get(frame[0])
        if cur == None:
            cur = zero_of(key_kind(frame[0]))
        s = binop("+", cur, frame[2])
        if s[1] != "":
            return s[1]
        cv = coerce(key_kind(frame[0]), s[0])
        if cv[1] != "":
            return cv[1]
        st["vars"][frame[0]] = cv[0]
        step = to_float(frame[2])
        value = to_float(cv[0])
        limit = to_float(frame[1])
        if (step >= 0.0 and value <= limit) or (step < 0.0 and value >= limit):
            load_line(st, frame[3])
            st["pos"] = frame[4]
            return ""
        st["fors"] = fors[:f]
    return done_stmt(st, toks, p)

# ON expr GOTO/GOSUB line,line,... picks the nth line (nothing happens if n is 0 or too big). Also ON ERROR GOTO line, which sets the error handler.
def st_on(st, toks, pos):
    if is_kw(toks, pos, "ERROR"):
        if not is_kw(toks, pos + 1, "GOTO"):
            return "SN"
        line = line_number(toks, pos + 2)
        if line < 0:
            return "SN"
        if line != 0 and line not in st["program"]:
            return "UL"
        st["onerr"] = line
```

```
        return done_stmt(st, toks, pos + 3)
    r = eval_at(st, toks, pos)
    if r[2] != "":
        return r[2]
    sel = number_arg(r[0])
    if sel[1] != "":
        return sel[1]
    p = r[1]
    if is_kw(toks, p, "GOTO"):
        sub = False
    elif is_kw(toks, p, "GOSUB"):
        sub = True
    else:
        return "SN"
    p += 1
    lines = []
    for _ in range(len(toks) + 1):
        ln = line_number(toks, p)
        if ln < 0:
            return "SN"
        lines.append(ln)
        p += 1
        if is_op(toks, p, ","):
            p += 1
        else:
            break
    if sel[0] < 0 or sel[0] > 255:
        return "FC"
    if sel[0] == 0 or sel[0] > len(lines):
        return done_stmt(st, toks, p)
    target = lines[sel[0] - 1]
    if sub:
        if target not in st["program"]:
            return "UL"
        st["gosub"].append((st["line"], p))
    return jump_to(st, target)

# INPUT ["prompt";] variables. Prints the prompt and leaves the machine waiting: `run` returns "waiting"
  and the host calls submit with the
# typed line, which feed_input handles. Only allowed inside a program ("ID" in direct mode).
def st_input(st, toks, pos):
    if st["line"] == 0:
        return "ID"
    p = pos
    prompt = "? "
    if p < len(toks) and toks[p][0] == "s":
        prompt = toks[p][1]
        p += 1
        if is_op(toks, p, ";"):
            prompt += "? "
            p += 1
        elif is_op(toks, p, ","):
            p += 1
        else:
            return "SN"
    targets = []
    for _ in range(len(toks) + 1):
        t = parse_target(st, toks, p)
        if t[3] != "":
            return t[3]
        targets.append((t[0], t[1]))
        p = t[2]
        if is_op(toks, p, ","):
            p += 1
        else:
            break
    if not at_end(toks, p):
        return "SN"
    st["pos"] = p
    st["waiting"] = {"targets": targets, "fields": [], "prompt": prompt}
    put_text(st, prompt)
    return ""

# Called with a typed line when the program is waiting at an INPUT.
def feed_input(st, text):
```

```
    w = st["waiting"]
    if w == None:
        return
    line = text.rstrip("\r\n")
    if st["echo"]:
        put_text(st, line + "\n")
    fields = w["fields"] + split_fields(line)
    targets = w["targets"]
    if len(fields) < len(targets):
        w["fields"] = fields
        put_text(st, "?? ")
        return
    if len(fields) > len(targets):
        put_text(st, "?EXTRA IGNORED\n")
    values = []
    for i in range(len(targets)):
        f = fields[i]
        if key_kind(targets[i][0]) == 3:
            values.append(f[0])
        else:
            r = parse_number_text(f[0]) if not f[1] else (False, 0)
            if not r[0]:
                put_text(st, "?REDO FROM START\n")
                w["fields"] = []
                put_text(st, w["prompt"])
                return
            values.append(r[1])
    st["waiting"] = None
    for i in range(len(targets)):
        err = assign(st, targets[i][0], targets[i][1], values[i])
        if err != "":
            handle_error(st, err)
            return

# READ variables: take the next DATA items, converting each to the variable's type. "OD" when the data
  runs out.
def st_read(st, toks, pos):
    p = pos
    for _ in range(len(toks) + 1):
        t = parse_target(st, toks, p)
        if t[3] != "":
            return t[3]
        if st["dptr"] >= len(st["data"]):
            return "OD"
        item = st["data"][st["dptr"]]
        st["dptr"] += 1
        if key_kind(t[0]) == 3:
            v = item[0]
        else:
            r = parse_number_text(item[0]) if not item[1] else (False, 0)
            if not r[0]:
                st["errat"] = item[2]   # Level II blames the DATA line, not the READ
                return "SN"
            v = r[1]
        err = assign(st, t[0], t[1], v)
        if err != "":
            return err
        p = t[2]
        if is_op(toks, p, ","):
            p += 1
        else:
            break
    return done_stmt(st, toks, p)

# DATA is only read by build_data; when execution reaches the statement it does nothing.
def st_data(st, toks, pos):
    return done_stmt(st, toks, pos + 1)

# RESTORE: read DATA again from the first item.
def st_restore(st, toks, pos):
    st["dptr"] = 0
    return done_stmt(st, toks, pos)

# DIM A(n[,m...]) ...: create arrays; each subscript runs from 0 to the number given. "DD" if the array
  exists already.
```

```
def st_dim(st, toks, pos):
    p = pos
    for _ in range(len(toks) + 1):
        if p >= len(toks) or toks[p][0] != "v" or not is_op(toks, p + 1, "("):
            return "SN"
        key = var_key(st, toks[p][1], toks[p][2])
        r = paren_args(st, toks, p + 1)
        if r[2] != "":
            return r[2]
        dims = []
        for d in r[0]:
            v = number_arg(d)
            if v[1] != "":
                return v[1]
            if v[0] < 0:
                return "FC"
            dims.append(v[0])
        if key in st["arrays"]:
            return "DD"
        err = new_array(st, key, dims)
        if err != "":
            return err
        p = r[1]
        if is_op(toks, p, ","):
            p += 1
        else:
            break
    return done_stmt(st, toks, p)

# REM: ignore the rest of the line.
def st_rem(st, toks, pos):
    return next_line(st)

# END: stop the program and show READY. It cannot be continued with CONT.
def st_end(st, toks, pos):
    st["running"] = False
    st["cont"] = None
    ready(st)
    return ""

# STOP: like BREAK, so CONT can carry on from here.
def st_stop(st, toks, pos):
    if st["col"] != 0:
        newline(st)
    put_text(st, "BREAK" + ((" IN %d" % st["line"]) if st["line"] != 0 else "") + "\n")
    st["cont"] = (st["line"], pos)
    stop(st)
    return ""

# CLS: clear the screen.
def st_cls(st, toks, pos):
    clear_screen(st)
    return done_stmt(st, toks, pos)

# SET(x,y): turn on a block-graphics dot (128 x 48 dots).
def st_set(st, toks, pos):
    return pixel(st, toks, pos, True)

# RESET(x,y): turn a block-graphics dot off.
def st_reset(st, toks, pos):
    return pixel(st, toks, pos, False)

# Shared by SET and RESET: check the two coordinates (0-127 and 0-47) and change the dot.
def pixel(st, toks, pos, on):
    r = paren_args(st, toks, pos)
    if r[2] != "":
        return r[2]
    if len(r[0]) != 2:
        return "SN"
    x = number_arg(r[0][0])
    y = number_arg(r[0][1])
    if x[1] != "" or y[1] != "":
        return "TM"
    if x[0] < 0 or x[0] > 127 or y[0] < 0 or y[0] > 47:
        return "FC"
```

```
    set_pixel(st, x[0], y[0], on)
    return done_stmt(st, toks, r[1])

# POKE address,value. Writes to video memory (addresses 15360 to 16383) change the screen; any other address is kept in `mem`, which PEEK reads back.
def st_poke(st, toks, pos):
    r1 = eval_at(st, toks, pos)
    if r1[2] != "":
        return r1[2]
    if not is_op(toks, r1[1], ","):
        return "SN"
    r2 = eval_at(st, toks, r1[1] + 1)
    if r2[2] != "":
        return r2[2]
    a = number_arg(r1[0])
    v = number_arg(r2[0])
    if a[1] != "" or v[1] != "":
        return "TM"
    addr = a[0]
    if addr < 0:
        addr += 65536
    if addr < 0 or addr > 65535 or v[0] < 0 or v[0] > 255:
        return "FC"
    if addr >= VIDEO_BASE and addr < VIDEO_BASE + CELLS:
        st["screen"][addr - VIDEO_BASE] = v[0]
    else:
        st["mem"][addr] = v[0]
    return done_stmt(st, toks, r2[1])

# RANDOM: reseed the random generator. Mixes in the step count so repeated calls differ, while a run from the same state stays repeatable.
def st_random(st, toks, pos):
    st["seed"] = (st["seed"] * 3 + 17 + st["steps"]) % 2147483648
    return done_stmt(st, toks, pos)

# TRON: from now on print each line number in brackets as it runs.
def st_tron(st, toks, pos):
    st["trace"] = True
    return done_stmt(st, toks, pos)

# TROFF: stop printing line numbers.
def st_troff(st, toks, pos):
    st["trace"] = False
    return done_stmt(st, toks, pos)

# Make the handler for DEFINT, DEFSNG, DEFDBL or DEFSTR. `kind` is the type (0 integer, 1 single, 2 double, 3 string) given to variables whose
# names start with the letters listed (A, or a range like A-F). Returns a function because the four statements differ only in `kind`.
def deftype_statement(kind):
    # The statement itself, closed over `kind`: read a list of single letters or ranges (A, B-F) and set their default type.
    def run_it(st, toks, pos):
        p = pos
        for _ in range(len(toks) + 1):
            if p >= len(toks) or toks[p][0] != "v" or len(toks[p][1]) != 1 or toks[p][2] != "":
                return "SN"
            lo = LETTERS.find(toks[p][1])
            hi = lo
            p += 1
            if is_op(toks, p, "-"):
                if p + 1 >= len(toks) or toks[p + 1][0] != "v" or len(toks[p + 1][1]) != 1:
                    return "SN"
                hi = LETTERS.find(toks[p + 1][1])
                p += 2
            for c in range(lo, hi + 1):
                st["deftype"][LETTERS[c]] = kind
            if is_op(toks, p, ","):
                p += 1
            else:
                break
        return done_stmt(st, toks, p)
    return run_it

# DEF FNname(params) = expression. The expression is compiled now and stored under the name; it is
```

```
    evaluated when called. Program lines only.
def st_def(st, toks, pos):
    if st["line"] == 0:
        return "ID"
    if not is_kw(toks, pos, "FN") or pos + 1 >= len(toks) or toks[pos + 1][0] != "v":
        return "SN"
    name = toks[pos + 1][1] + toks[pos + 1][2]
    p = pos + 2
    params = []
    if is_op(toks, p, "("):
        p += 1
        for _ in range(len(toks) + 1):
            if p >= len(toks) or toks[p][0] != "v":
                return "SN"
            params.append(var_key(st, toks[p][1], toks[p][2]))
            p += 1
            if is_op(toks, p, ","):
                p += 1
            else:
                break
        if not is_op(toks, p, ")"):
            return "SN"
        p += 1
    if not is_op(toks, p, "="):
        return "SN"
    r = compile_expr(toks, p + 1)
    if r[2] != "":
        return r[2]
    st["fns"][name] = (params, r[0])
    return done_stmt(st, toks, r[1])

# CLEAR: forget all variables, arrays and loop and subroutine state. The optional number (string space in
    BASIC) is evaluated and ignored.
def st_clear(st, toks, pos):
    p = pos
    if not at_end(toks, p):
        r = eval_at(st, toks, p)
        if r[2] != "":
            return r[2]
        p = r[1]
    st["vars"] = {}
    st["arrays"] = {}
    st["gosub"] = []
    st["fors"] = []
    return done_stmt(st, toks, p)

# ERROR n: raise error number n as if it had happened. 1 to 24 are the standard codes; larger ones are
    "UE" (unprintable error).
def st_error(st, toks, pos):
    r = eval_at(st, toks, pos)
    if r[2] != "":
        return r[2]
    c = number_arg(r[0])
    if c[1] != "":
        return c[1]
    if c[0] < 1 or c[0] > 255:
        return "FC"
    if c[0] <= len(ERRORS):
        return ERRORS[c[0] - 1]
    return "UE"

# RESUME [NEXT | line]: leave the ON ERROR handler. Plain RESUME retries the statement that failed, NEXT
    carries on after it, and a line number
# goes there. "RW" if no error is being handled.
def st_resume(st, toks, pos):
    if not st["inerr"]:
        return "RW"
    where = st["resume"]
    st["inerr"] = False
    p = pos
    if is_kw(toks, p, "NEXT"):
        load_line(st, where[0])
        t = st["toks"]
        q = where[1]
        found = len(t)
```

```
        for _ in range(len(t) + 1):
            if q >= len(t):
                break
            if t[q] == ("o", ":"):
                found = q + 1
                break
            q += 1
        st["pos"] = found
        return ""
    ln = line_number(toks, p)
    if ln > 0:
        return jump_to(st, ln)
    load_line(st, where[0])
    st["pos"] = where[1]
    return ""

# ------------------------------------------------------------------ commands (also usable as statements)

# What RUN and NEW clear: variables, arrays, user functions, the control stacks, the error handler, type defaults. Then rebuild the DATA list.
def reset_run_state(st):
    st["vars"] = {}
    st["arrays"] = {}
    st["fns"] = {}
    st["gosub"] = []
    st["fors"] = []
    st["onerr"] = 0
    st["inerr"] = False
    st["waiting"] = None
    st["cont"] = None
    st["deftype"] = {c: 1 for c in LETTERS.elems()}
    build_data(st)

# RUN [line]: clear the run state and start at the first line (or the given one).
def st_run(st, toks, pos):
    ensure_index(st)
    reset_run_state(st)
    if len(st["order"]) == 0:
        stop(st)
        return ""
    start = st["order"][0]
    if pos < len(toks) and toks[pos][0] == "n":
        start = line_number(toks, pos)
    return goto_line(st, start)

# EDIT n: start editing line n. From here until the editor finishes, every key typed goes to edit_key.
def st_edit(st, toks, pos):
    if st["line"] != 0:
        return "ID"     # EDIT takes over the keyboard, so it is a direct command only
    line = line_number(toks, pos)
    if line < 0 or not at_end(toks, pos + 1):
        return "SN"
    if line not in st["program"]:
        return "UL"
    st["edit"] = new_edit(line, display_text(st["program"][line]))
    put_text(st, "%d " % line)
    st["running"] = False
    return ""

# One key for the line editor. Returns "edit" while the editor is still open, otherwise "done".
def edit_key(st, key):
    e = st["edit"]
    if e == None:
        return "done"
    r = edit_step(e, key)
    if r[0] != "":
        put_text(st, r[0])
    if r[1] == "":
        return "edit"
    st["edit"] = None
    if r[1] == "save":
        text = edit_text(e)
        if text.strip() == "":
            st["program"].pop(e["line"], None)
        else:
```

```
            st["program"][e["line"]] = text
        st["tokcache"].pop(e["line"], None)
        st["rpn"] = {}
        st["dirty"] = True
        st["cont"] = None
    put_text(st, "\n")
    prompt_again(st)
    return "done"

# Whether the editor has the keyboard.
def editing(st):
    return st["edit"] != None

# NEW: delete the program and everything else.
def st_new(st, toks, pos):
    st["program"] = {}
    st["tokcache"] = {}
    st["rpn"] = {}
    st["dirty"] = True
    reset_run_state(st)
    stop(st)
    return ""

# CONT: carry on after BREAK or STOP. "CN" if the program changed or ended, since there is nothing to continue.
def st_cont(st, toks, pos):
    if st["cont"] == None:
        return "CN"
    where = st["cont"]
    st["cont"] = None
    if where[0] != 0 and where[0] not in st["program"]:
        return "CN"
    load_line(st, where[0])
    st["pos"] = where[1]
    st["running"] = True
    return ""

# Parse an optional "n", "n-m", "-m" or "n-" after LIST or DELETE. Returns (low, high, error).
def line_range(toks, pos):
    lo = 0
    hi = 65535
    p = pos
    if p < len(toks) and toks[p][0] == "n":
        lo = line_number(toks, p)
        hi = lo
        p += 1
        if is_op(toks, p, "-"):
            hi = 65535
            p += 1
            if p < len(toks) and toks[p][0] == "n":
                hi = line_number(toks, p)
                p += 1
    elif is_op(toks, p, "-"):
        p += 1
        if p < len(toks) and toks[p][0] == "n":
            hi = line_number(toks, p)
            p += 1
        else:
            return (0, 0, "SN")
    if not at_end(toks, p):
        return (0, 0, "SN")
    return (lo, hi, "")

# Keywords and names in upper case, but text in quotes, after REM and in DATA exactly as typed.
def display_text(text):
    out = ""
    in_quotes = False
    n = len(text)
    i = 0
    u = text.upper()
    for _ in range(n + 1):
        if i >= n:
            break
        c = text[i]
        if c == "\"":
```

```
            in_quotes = not in_quotes
            out += c
            i += 1
        elif in_quotes:
            out += c
            i += 1
        elif u.startswith("REM", i) or c == "'":
            out += u[i:i + 3] + text[i + 3:] if c != "'" else text[i:]
            i = n
        elif u.startswith("DATA", i):
            end = n
            q = False
            for k in range(i + 4, n):
                if text[k] == "\"":
                    q = not q
                elif text[k] == ":" and not q:
                    end = k
                    break
            out += "DATA" + text[i + 4:end]
            i = end
        else:
            out += u[i]
            i += 1
    return out

# LIST [range]: print the program lines in order.
def st_list(st, toks, pos):
    r = line_range(toks, pos)
    if r[2] != "":
        return r[2]
    ensure_index(st)
    for line in st["order"]:
        if line >= r[0] and line <= r[1]:
            put_text(st, "%d %s\n" % (line, display_text(st["program"][line])))
    stop(st)
    return ""

# DELETE range: remove the lines. "FC" if the range holds no lines.
def st_delete(st, toks, pos):
    r = line_range(toks, pos)
    if r[2] != "":
        return r[2]
    removed = [l for l in st["program"].keys() if l >= r[0] and l <= r[1]]
    if len(removed) == 0:
        return "FC"
    for l in removed:
        st["program"].pop(l)
        st["tokcache"].pop(l, None)
    st["rpn"] = {}
    st["dirty"] = True
    stop(st)
    return ""

# Statement keyword -> the function that runs it. exec_stmt looks the first keyword of a statement up here. A statement starting with a
# variable name is an assignment and does not need an entry.
STATEMENTS = {
    "PRINT": st_print, "GOTO": st_goto, "GOSUB": st_gosub, "RETURN": st_return, "IF": st_if, "ELSE": st_else,
    "FOR": st_for, "NEXT": st_next, "ON": st_on, "INPUT": st_input, "READ": st_read, "DATA": st_data,
    "RESTORE": st_restore, "DIM": st_dim, "REM": st_rem, "END": st_end, "STOP": st_stop, "CLS": st_cls, "SET": st_set,
    "RESET": st_reset, "POKE": st_poke, "RANDOM": st_random, "TRON": st_tron, "TROFF": st_troff, "DEF": st_def,
    "DEFINT": deftype_statement(0), "DEFSNG": deftype_statement(1), "DEFDBL": deftype_statement(2),
    "DEFSTR": deftype_statement(3), "CLEAR": st_clear, "ERROR": st_error, "RESUME": st_resume, "RUN": st_run,
    "EDIT": st_edit, "NEW": st_new, "CONT": st_cont, "LIST": st_list, "DELETE": st_delete, "LET": st_let,
}

# ---------------------------------------------------------------- running

# Execute one statement (or step past the end of a line). Returns an error code or "".
def exec_stmt(st):
    toks = st["toks"]
```

```
    pos = st["pos"]
    if pos >= len(toks):
        if st["tokerr"] != "":
            return st["tokerr"]
        return next_line(st)
    t = toks[pos]
    if t[0] == "o" and t[1] == ":":
        st["pos"] = pos + 1
        return ""
    st["stmt_start"] = (st["line"], pos)
    if t[0] == "v":
        return st_let(st, toks, pos)
    if t[0] == "k":
        handler = STATEMENTS.get(t[1])
        if handler != None:
            if t[1] == "LET":
                return st_let(st, toks, pos + 1)
            return handler(st, toks, pos + 1)
    return "SN"

# Run up to `budget` statements. Returns "done" (back at the READY prompt), "waiting" (at an INPUT) or
  "budget" (more to do).
def run(st, budget):
    for _ in range(budget):
        if not st["running"] or st["waiting"] != None:
            break
        st["steps"] += 1
        err = exec_stmt(st)
        if err != "":
            handle_error(st, err)
    if st["waiting"] != None:
        return "waiting"
    if st["running"]:
        return "budget"
    return "done"

# Deal with one line typed at the keyboard: a program line is stored, anything else is run at once.
def enter_line(st, text):
    s = text.rstrip("\r\n").lstrip()
    if st["echo"]:
        put_text(st, s + "\n")
    if s == "":
        prompt_again(st)    # a bare Enter gets a fresh prompt, as on the real machine
        return
    k = span_digits(s, 0)
    if k > 0:
        num = int(s[:k])
        rest = s[k:].lstrip()
        if num > 65529:
            st["direct"] = ([], "")
            st["line"] = 0
            handle_error(st, "SN")
            return
        if rest == "":
            st["program"].pop(num, None)
            st["tokcache"].pop(num, None)
        else:
            st["program"][num] = rest
            st["tokcache"].pop(num, None)
        st["rpn"] = {}
        st["dirty"] = True
        st["cont"] = None
        prompt_again(st)    # storing a program line shows no READY, only a new prompt
        return
    toks = tokenize(s)
    st["direct"] = toks
    st["line"] = 0
    load_line(st, 0)
    st["pos"] = 0
    st["running"] = True

# Entry point for a typed line: answer a waiting INPUT, or enter a command or program line.
def submit(st, text, budget):
    if st["waiting"] != None:
        feed_input(st, text)
```

```
    elif not st["running"]:
        enter_line(st, text)
    return run(st, budget)

# After a fault in this script itself (a bug, not a BASIC error): drop whatever was running and show READY again. The program,
# variables and screen are left as they were.
def recover(st):
    st["running"] = False
    st["waiting"] = None
    st["cont"] = None
    ready(st)
    return "done"

# Whether typed lines are echoed into the output. A terminal shows what the user types by itself, so a terminal host turns this off;
# a test or a pipe has no echo of its own, so it stays on.
def set_echo(st, on):
    st["echo"] = on

# Queue keys for INKEY$.
def press_keys(st, keys):
    for ch in keys.elems():
        st["inkey"].append(ch)
```

expr.star

Expressions: compiled to reverse Polish notation, then evaluated, with no recursion

```
# Expressions for the BASIC interpreter: compile a token list to reverse Polish notation, then evaluate it.
#
# Starlark does not allow recursion, so neither step recurses. The compiler is the shunting-yard algorithm with an operator
# stack, and the evaluator is a loop over the compiled items with an explicit stack of frames for DEF FN calls. Nesting of
# parentheses, of functions and of FN calls is therefore limited by data, never by the host's stack.
#
# Compiled items (the "rpn" list):
#   ("c", value)                push a constant
#   ("var", name, suffix)       push a scalar variable
#   ("arr", (name, suffix), n)  pop n subscripts, push an array element
#   ("B", op)  ("U", op)        binary and unary operator
#   ("call", NAME, n)           built-in function with n arguments
#   ("ufn", NAME, n)            user function (DEF FN) with n arguments
#
# Every function that can fail returns its error as a two letter BASIC code ("TM", "OV", "FC", "/0", ...), "" for none.

load("numbers.star",
     "INT_MAX", "INT_MIN", "kind_of", "to_float", "floor_int", "negate", "to_single", "overflows", "str_text", "val_text",
     "f_sqr", "f_exp", "f_log", "f_sin", "f_cos", "f_atn", "f_pow", "rnd_next")

# Character -> code (0 to 255), for ASC. Characters outside this table read as 63, "?", since Level II had no such character.
CODES = {"%c" % i: i for i in range(256)}

# The type suffix of a variable name -> its kind: 0 integer (%), 1 single precision (!), 2 double precision (#), 3 string ($).
SUFFIX_KIND = {"%": 0, "!": 1, "#": 2, "$": 3}
# The reverse: kind number -> suffix character. Used to give an untyped name its default type.
KIND_SUFFIX = ["%", "!", "#", "$"]

# Names that are built-in functions. A set, so the compiler can ask "is this word a function?" in one lookup.
FUNCS = {}
for _w in ["SGN", "INT", "ABS", "FRE", "INP", "POS", "SQR", "RND", "LOG", "EXP", "COS", "SIN", "TAN", "ATN", "PEEK", "CINT",
           "CSNG", "CDBL", "FIX", "LEN", "STR$", "VAL", "ASC", "CHR$", "LEFT$", "RIGHT$", "MID$", "INSTR", "STRING$",
           "POINT", "MEM", "INKEY$", "ERL", "ERR"]:
    FUNCS[_w] = True

# Functions written without parentheses (INKEY$, MEM, ERL, ERR).
ZERO_ARG = {"INKEY$": True, "MEM": True, "ERL": True, "ERR": True}

# Operator precedence for the shunting-yard compiler: a higher number binds tighter. "u-" is unary minus. As on Level II, ^ binds tighter than
# unary minus, so -2^2 is -4.
PREC = {"OR": 1, "AND": 2, "NOT": 3, "=": 4, "<>": 4, "<": 4, ">": 4, "<=": 4, ">=": 4,
        "+": 5, "-": 5, "*": 6, "/": 6, "u-": 7, "^": 8}

# The symbolic binary operators. AND and OR are keywords and are handled separately.
BINARY = {"+": True, "-": True, "*": True, "/": True, "^": True,
          "=": True, "<>": True, "<": True, ">": True, "<=": True, ">=": True}

# The longest string BASIC allows; a longer result is the "LS" (string too long) error.
MAX_STRING = 255

# The dictionary key for a variable: its two-character name plus a type suffix. An untyped name takes its type from the DEFINT/DEFSTR table
# for its first letter, so with the default (single precision) A is stored as "A!".
def var_key(st, name, suffix):
    if suffix == "":
        return name + KIND_SUFFIX[st["deftype"][name[0]]]
    return name + suffix

# The value a variable of this kind has before it is assigned: 0, 0.0, a double 0.0, or the empty string.
def zero_of(kind):
```

```
    if kind == 0:
        return 0
    if kind == 1:
        return 0.0
    if kind == 2:
        return (0.0,)
    return ""

# The kind (0 to 3) of a variable key, from its suffix.
def key_kind(key):
    return SUFFIX_KIND[key[-1]]

# Convert a value to what a variable of this kind holds. Returns (value, error).
def coerce(kind, v):
    vk = kind_of(v)
    if kind == 3:
        if vk != 3:
            return (None, "TM")
        return (v, "")
    if vk == 3:
        return (None, "TM")
    if kind == vk:
        return (v, "")
    f = to_float(v)
    if kind == 0:
        n = floor_int(f)
        if n < INT_MIN or n > INT_MAX:
            return (None, "OV")
        return (n, "")
    if kind == 1:
        if overflows(f):
            return (None, "OV")
        return (to_single(f), "")
    return ((f,), "")

# ---------------------------------------------------------------- compiling

# Compile the expression that starts at toks[pos]. Stops at the first token that cannot continue it (a comma, semicolon,
# colon, THEN, TO, STEP, ELSE, or an unmatched close parenthesis). Returns (rpn, next position, error).
def compile_expr(toks, pos):
    # The output (the compiled RPN list), the operator stack, and one argument counter per open "(" that belongs to a function call or array.
    out = []
    ops = []
    argc = []
    # `expect` is True when an operand (a value, "(", a function, NOT or a minus sign) must come next, and False after an operand, when an operator
    # or a closing bracket must. It is what tells unary minus from binary minus, and catches errors like "3 4" and "3 +".
    expect = True
    # The loop is bounded by the number of tokens (each turn consumes at least one) because Starlark has no `while`.
    n = len(toks)
    i = pos
    depth = 0
    for _ in range(n + 2):
        if i >= n:
            break
        t = toks[i]
        kind = t[0]
        # A number or a string: push it as a constant. Two operands in a row means the expression has ended, so stop and let the caller deal with the rest.
        if kind == "n" or kind == "s":
            if not expect:
                break
            out.append(("c", t[1]))
            expect = False
            i += 1
        # A variable. Followed by "(" it is an array element: mark the open bracket on the operator stack so the subscripts are gathered, and count
        # its first argument. Otherwise it is a plain variable.
        elif kind == "v":
            if not expect:
                break
```

```
            if i + 1 < n and toks[i + 1][0] == "o" and toks[i + 1][1] == "(":
                ops.append(("M", "arr", (t[1], t[2])))
                argc.append(1)
                depth += 1
                i += 2
            else:
                out.append(("var", t[1], t[2]))
                expect = False
                i += 1
        # A keyword: a built-in function, FN (a user function), NOT, AND or OR. Any other keyword (THEN, TO, STEP, ELSE...) ends the expression.
        elif kind == "k":
            w = t[1]
            if w in FUNCS:
                if not expect:
                    break
                if i + 1 < n and toks[i + 1][0] == "o" and toks[i + 1][1] == "(":
                    ops.append(("M", "call", w))
                    argc.append(1)
                    depth += 1
                    i += 2
                elif w in ZERO_ARG:
                    out.append(("call", w, 0))
                    expect = False
                    i += 1
                else:
                    return ([], i, "SN")
            elif w == "FN":
                if not expect:
                    break
                if i + 1 >= n or toks[i + 1][0] != "v":
                    return ([], i, "SN")
                name = toks[i + 1][1] + toks[i + 1][2]
                if i + 2 < n and toks[i + 2][0] == "o" and toks[i + 2][1] == "(":
                    ops.append(("M", "ufn", name))
                    argc.append(1)
                    depth += 1
                    i += 3
                else:
                    out.append(("ufn", name, 0))
                    expect = False
                    i += 2
            # NOT is a prefix operator and applies to what follows it, so it just goes on the operator stack.
            elif w == "NOT":
                if not expect:
                    break
                ops.append(("U", "NOT"))
                i += 1
            # Binary operators: first move every operator already on the stack that binds at least as tightly into the output, then push this one.
            elif w == "AND" or w == "OR":
                if expect:
                    return ([], i, "SN")
                p = PREC[w]
                for _ in range(len(ops)):
                    if len(ops) == 0 or ops[-1][0] == "M" or PREC[ops[-1][1]] < p:
                        break
                    out.append(ops.pop())
                ops.append(("B", w))
                expect = True
                i += 1
            else:
                break
        # An operator or punctuation mark.
        elif kind == "o":
            op = t[1]
            if op == "(":
                if not expect:
                    break
                ops.append(("M", "paren", ""))
                depth += 1
                i += 1
            # A closing bracket: move operators to the output back to the matching "(" and drop it. If that bracket belonged to a function, array or FN,
```

```
            # emit that item with its argument count.
            elif op == ")":
                if depth == 0:
                    break
                if expect:
                    return ([], i, "SN")
                for _ in range(len(ops) + 1):
                    if ops[-1][0] == "M":
                        break
                    out.append(ops.pop())
                m = ops.pop()
                depth -= 1
                if m[1] == "arr":
                    out.append(("arr", m[2], argc.pop()))
                elif m[1] == "call":
                    out.append(("call", m[2], argc.pop()))
                elif m[1] == "ufn":
                    out.append(("ufn", m[2], argc.pop()))
                expect = False
                i += 1
            # An argument separator inside a function call or subscript list: finish the argument (flush its operators) and count the next one.
            elif op == ",":
                if depth == 0:
                    break
                if expect:
                    return ([], i, "SN")
                for _ in range(len(ops) + 1):
                    if ops[-1][0] == "M":
                        break
                    out.append(ops.pop())
                if ops[-1][1] == "paren":
                    return ([], i, "SN")
                argc[-1] += 1
                expect = True
                i += 1
            # A sign where an operand is expected. A minus becomes the unary operator; a plus does nothing.
            elif expect and (op == "-" or op == "+"):
                if op == "-":
                    ops.append(("U", "u-"))
                i += 1
            # An ordinary binary operator: pop everything that binds tighter or equally, then push this one.
            elif op in BINARY:
                if expect:
                    return ([], i, "SN")
                p = PREC[op]
                for _ in range(len(ops)):
                    if len(ops) == 0 or ops[-1][0] == "M" or PREC[ops[-1][1]] < p:
                        break
                    out.append(ops.pop())
                ops.append(("B", op))
                expect = True
                i += 1
            else:
                break
        else:
            break
    # Ended in the middle of an expression (an operator or "(" still open): a syntax error.
    if expect or depth != 0:
        return ([], i, "SN")
    # End of the expression: move the remaining operators to the output.
    for _ in range(len(ops)):
        if len(ops) == 0:
            break
        out.append(ops.pop())
    return (out, i, "")

# ---------------------------------------------------------------- operators

# A number as a 16-bit integer, rounding down. Returns (value, error): "TM" for a string, "OV" outside -32768 to 32767. Used by AND, OR and NOT,
# which work on 16-bit integers.
def to_int16(v):
    if kind_of(v) == 3:
```

```
        return (0, "TM")
    n = floor_int(to_float(v))
    if n < INT_MIN or n > INT_MAX:
        return (0, "OV")
    return (n, "")

# Check a floating-point result and give it the right type: a double stays a double, anything else is rounded to single precision. Returns
# (value, error); a result that overflows (or is not a number) is the "OV" error.
def real_result(r, double):
    if double:
        if r > 1.7976931348623157e308 or r < -1.7976931348623157e308 or r != r:
            return (None, "OV")
        return ((r,), "")
    if overflows(r):
        return (None, "OV")
    return (to_single(r), "")

# Apply a comparison operator. BASIC's true is -1 and false is 0, so the result is -1 or 0.
def compare(op, a, b):
    if op == "=":
        r = a == b
    elif op == "<>":
        r = a != b
    elif op == "<":
        r = a < b
    elif op == ">":
        r = a > b
    elif op == "<=":
        r = a <= b
    else:
        r = a >= b
    if r:
        return (-1, "")
    return (0, "")

# Apply a binary operator to two values. Returns (value, error). Strings may be joined with + and compared; AND/OR work on 16-bit integers;
# integer + - * stay integers while they fit and become single precision when they overflow; mixing types widens to the larger one; division
# by zero is "/0".
def binop(op, a, b):
    ka = kind_of(a)
    kb = kind_of(b)
    if ka == 3 or kb == 3:
        if ka != kb:
            return (None, "TM")
        if op == "+":
            r = a + b
            if len(r) > MAX_STRING:
                return (None, "LS")
            return (r, "")
        if op in ["=", "<>", "<", ">", "<=", ">="]:
            return compare(op, a, b)
        return (None, "TM")
    if op == "AND" or op == "OR":
        x = to_int16(a)
        y = to_int16(b)
        if x[1] != "":
            return (None, x[1])
        if y[1] != "":
            return (None, y[1])
        if op == "AND":
            return (x[0] & y[0], "")
        return (x[0] | y[0], "")
    k = max(ka, kb)
    if op in ["=", "<>", "<", ">", "<=", ">="]:
        return compare(op, to_float(a) if k > 0 else a, to_float(b) if k > 0 else b)
    if k == 0 and (op == "+" or op == "-" or op == "*"):
        if op == "+":
            r = a + b
        elif op == "-":
            r = a - b
        else:
            r = a * b
```

```
        if r >= INT_MIN and r <= INT_MAX:
            return (r, "")
        return real_result(float(r), False)
    fa = to_float(a)
    fb = to_float(b)
    double = k == 2
    if op == "+":
        return real_result(fa + fb, double)
    if op == "-":
        return real_result(fa - fb, double)
    if op == "*":
        return real_result(fa * fb, double)
    if op == "/":
        if fb == 0.0:
            return (None, "/0")
        return real_result(fa / fb, double)
    if op == "^":
        r = f_pow(fa, fb)
        if r[1] != "":
            return (None, r[1])
        return real_result(r[0], double)
    return (None, "SN")

# Apply a unary operator: negation, or NOT, which is the 16-bit complement (-x - 1). Returns (value, error).
def unop(op, a):
    if kind_of(a) == 3:
        return (None, "TM")
    if op == "u-":
        return (negate(a), "")
    x = to_int16(a)
    if x[1] != "":
        return (None, x[1])
    return (-x[0] - 1, "")

# ------------------------------------------------------------------ built-in functions

# Give a math function's result the right type (single, or double if the argument was a double). Same as real_result.
def real_of(x, double):
    return real_result(x, double)

# A whole number as BASIC's integer type if it fits in 16 bits, otherwise as a single-precision number.
def int_or_single(n):
    if n >= INT_MIN and n <= INT_MAX:
        return (n, "")
    return real_result(float(n), False)

# True if `v` is a string (kind 3). Used to check function arguments.
def need_string(v):
    return kind_of(v) == 3

# Call built-in `name` with the already evaluated arguments. Returns (value, error).
def call_function(st, name, args):
    n = len(args)
    a = args[0] if n > 0 else None
    if name == "LEN":
        if n != 1 or not need_string(a):
            return (None, "TM")
        return (len(a), "")
    if name == "LEFT$" or name == "RIGHT$":
        if n != 2 or not need_string(a) or kind_of(args[1]) == 3:
            return (None, "TM")
        c = floor_int(to_float(args[1]))
        if c < 0 or c > 255:
            return (None, "FC")
        if name == "LEFT$":
            return (a[:c], "")
        if c >= len(a):
            return (a, "")
        return (a[len(a) - c:], "")
    if name == "MID$":
        if (n != 2 and n != 3) or not need_string(a) or kind_of(args[1]) == 3:
            return (None, "TM")
        start = floor_int(to_float(args[1]))
```

```
        if start < 1 or start > 255:
            return (None, "FC")
        if n == 3:
            if kind_of(args[2]) == 3:
                return (None, "TM")
            c = floor_int(to_float(args[2]))
            if c < 0 or c > 255:
                return (None, "FC")
            return (a[start - 1:start - 1 + c], "")
        return (a[start - 1:], "")
    if name == "ASC":
        if n != 1 or not need_string(a):
            return (None, "TM")
        if len(a) == 0:
            return (None, "FC")
        return (CODES.get(a[0], 63), "")
    if name == "CHR$":
        if n != 1 or kind_of(a) == 3:
            return (None, "TM")
        c = floor_int(to_float(a))
        if c < 0 or c > 255:
            return (None, "FC")
        return ("%c" % c, "")
    if name == "STR$":
        if n != 1 or kind_of(a) == 3:
            return (None, "TM")
        return (str_text(a), "")
    if name == "VAL":
        if n != 1 or not need_string(a):
            return (None, "TM")
        return (val_text(a), "")
    if name == "INSTR":
        if n != 2 and n != 3:
            return (None, "SN")
        start = 1
        s = args[0]
        t = args[1]
        if n == 3:
            if kind_of(args[0]) == 3:
                return (None, "TM")
            start = floor_int(to_float(args[0]))
            if start < 1 or start > 255:
                return (None, "FC")
            s = args[1]
            t = args[2]
        if not need_string(s) or not need_string(t):
            return (None, "TM")
        if start > len(s):
            return (0, "")
        if t == "":
            return (start, "")
        return (s.find(t, start - 1) + 1, "")
    if name == "STRING$":
        if n != 2 or kind_of(a) == 3:
            return (None, "TM")
        c = floor_int(to_float(a))
        if c < 0 or c > 255:
            return (None, "FC")
        ch = args[1]
        if need_string(ch):
            if len(ch) == 0:
                return (None, "FC")
            return (ch[0] * c, "")
        code = floor_int(to_float(ch))
        if code < 0 or code > 255:
            return (None, "FC")
        return (("%c" % code) * c, "")
    if name == "INKEY$":
        if len(st["inkey"]) > 0:
            return (st["inkey"].pop(0), "")
        return ("", "")
    if name == "MEM" or name == "FRE":
        return (14000, "")
    if name == "ERL":
        return int_or_single(st["errline"])
```

```
    if name == "ERR":
        return (st["errcode"], "")
    if name == "POS":
        return (st["col"], "")
    if name == "POINT":
        if n != 2:
            return (None, "SN")
        x = floor_int(to_float(args[0]))
        y = floor_int(to_float(args[1]))
        if x < 0 or x > 127 or y < 0 or y > 47:
            return (None, "FC")
        c = st["screen"][(y // 3) * 64 + x // 2]
        if c >= 128 and c <= 191 and (c & (1 << ((y % 3) * 2 + x % 2))) != 0:
            return (1, "")
        return (0, "")
    if name == "PEEK" or name == "INP":
        if n != 1 or kind_of(a) == 3:
            return (None, "TM")
        addr = floor_int(to_float(a))
        if addr < 0:
            addr += 65536
        if addr >= 15360 and addr < 16384:
            return (st["screen"][addr - 15360], "")
        return (st["mem"].get(addr, 0), "")
    if n != 1:
        return (None, "SN")
    if kind_of(a) == 3:
        return (None, "TM")
    k = kind_of(a)
    double = k == 2
    x = to_float(a)
    if name == "ABS":
        if k == 0:
            return int_or_single(abs(a))
        return (negate(a) if x < 0.0 else a, "")
    if name == "SGN":
        if x > 0.0:
            return (1, "")
        if x < 0.0:
            return (-1, "")
        return (0, "")
    if name == "INT":
        if k == 0:
            return (a, "")
        f = floor_int(x)
        if double:
            return ((float(f),), "")
        return int_or_single(f)
    if name == "FIX":
        if k == 0:
            return (a, "")
        f = int(x)
        if double:
            return ((float(f),), "")
        return int_or_single(f)
    if name == "CINT":
        f = floor_int(x)
        if f < INT_MIN or f > INT_MAX:
            return (None, "OV")
        return (f, "")
    if name == "CSNG":
        return real_of(x, False)
    if name == "CDBL":
        return ((x,), "")
    if name == "SQR":
        if x < 0.0:
            return (None, "FC")
        return real_of(f_sqr(x), double)
    if name == "EXP":
        r = f_exp(x)
        if r == None:
            return (None, "OV")
        return real_of(r, double)
    if name == "LOG":
        if x <= 0.0:
```

```
            return (None, "FC")
        return real_of(f_log(x), double)
    if name == "SIN":
        return real_of(f_sin(x), double)
    if name == "COS":
        return real_of(f_cos(x), double)
    if name == "TAN":
        c = f_cos(x)
        if c == 0.0:
            return (None, "OV")
        return real_of(f_sin(x) / c, double)
    if name == "ATN":
        return real_of(f_atn(x), double)
    if name == "RND":
        if x < 0.0:
            st["seed"] = (floor_int(-x) * 7919 + 13) % 2147483648
        st["seed"] = rnd_next(st["seed"])
        frac = (st["seed"] // 128) / 16777216.0
        if x >= 1.0:
            return int_or_single(int(frac * floor_int(x)) + 1)
        return (to_single(frac), "")
    return (None, "SN")

# ------------------------------------------------------------------ arrays

# Create an array. `dims` holds the largest subscript of each dimension, so the element count is the product of (d + 1). "OM" (out of memory)
# if that is more than 20,000 elements. Every element starts at the zero value of the array's type.
def new_array(st, key, dims):
    total = 1
    for d in dims:
        total *= d + 1
    if total > 20000:
        return "OM"
    st["arrays"][key] = [dims, [zero_of(key_kind(key))] * total]
    return ""

# Flat position of an element, creating the array (with 10 as every bound) on first use. Returns (array, position, error).
def locate(st, key, subs):
    arr = st["arrays"].get(key)
    if arr == None:
        err = new_array(st, key, [10] * len(subs))
        if err != "":
            return (None, 0, err)
        arr = st["arrays"][key]
    dims = arr[0]
    if len(dims) != len(subs):
        return (None, 0, "BS")
    pos = 0
    for j in range(len(dims)):
        s = subs[j]
        if kind_of(s) == 3:
            return (None, 0, "TM")
        s = floor_int(to_float(s))
        if s < 0 or s > dims[j]:
            return (None, 0, "BS")
        pos = pos * (dims[j] + 1) + s
    return (arr, pos, "")

# ------------------------------------------------------------------ evaluating

MAX_FN_DEPTH = 30

# Run a compiled expression. Returns (value, error).
def eval_rpn(st, rpn):
    stack = []
    frames = []
    cur = rpn
    pc = 0
    # A bound, not a limit anyone should reach: Starlark has no `while`, and each turn takes one item.
    for _ in range(100000000):
        # Reached the end of the current code. Either the whole expression is done (no frames left), or a user function has finished: restore the
        # caller's variables, code position and stack, and push the function's value for the caller to use.
```

```
        if pc >= len(cur):
            val = stack[-1]
            if len(frames) == 0:
                return (val, "")
            f = frames.pop()
            for pair in f[3]:
                if pair[1] == None:
                    st["vars"].pop(pair[0], None)
                else:
                    st["vars"][pair[0]] = pair[1]
            cur = f[0]
            pc = f[1]
            stack = f[2]
            stack.append(val)
            continue
        item = cur[pc]
        pc += 1
        tag = item[0]
        if tag == "c":
            stack.append(item[1])
        elif tag == "var":
            key = var_key(st, item[1], item[2])
            v = st["vars"].get(key)
            if v == None:
                v = zero_of(key_kind(key))
            stack.append(v)
        # A binary operator: its two operands are the top of the stack (right operand on top).
        elif tag == "B":
            b = stack.pop()
            a = stack.pop()
            r = binop(item[1], a, b)
            if r[1] != "":
                return (None, r[1])
            stack.append(r[0])
        elif tag == "U":
            r = unop(item[1], stack.pop())
            if r[1] != "":
                return (None, r[1])
            stack.append(r[0])
        # A built-in function: take its arguments off the stack, in order, and call it.
        elif tag == "call":
            c = item[2]
            args = stack[len(stack) - c:]
            stack = stack[:len(stack) - c]
            r = call_function(st, item[1], args)
            if r[1] != "":
                return (None, r[1])
            stack.append(r[0])
        # An array element: take the subscripts off the stack and look up the element (creating a
10-element array on first use, as BASIC does).
        elif tag == "arr":
            c = item[2]
            subs = stack[len(stack) - c:]
            stack = stack[:len(stack) - c]
            key = var_key(st, item[1][0], item[1][1])
            loc = locate(st, key, subs)
            if loc[2] != "":
                return (None, loc[2])
            stack.append(loc[0][1][loc[1]])
        else:
            c = item[2]
            args = stack[len(stack) - c:]
            stack = stack[:len(stack) - c]
            # Anything else is a call to a DEF FN function.
            defn = st["fns"].get(item[1])
            if defn == None:
                return (None, "UF")
            params = defn[0]
            if len(params) != len(args):
                return (None, "SN")
            # Functions calling functions are limited to 30 deep ("OM"), since each level costs a frame.
            if len(frames) >= MAX_FN_DEPTH:
                return (None, "OM")
            # The values the parameters hide are saved so they can be restored afterwards. None means the
variable did not exist.
```

```
            saved = []
            for j in range(len(params)):
                cv = coerce(key_kind(params[j]), args[j])
                if cv[1] != "":
                    return (None, cv[1])
                saved.append((params[j], st["vars"].get(params[j])))
            for j in range(len(params)):
                st["vars"][params[j]] = coerce(key_kind(params[j]), args[j])[0]
            # Remember where to come back to, then switch to running the function's body.
            frames.append((cur, pc, stack, saved))
            cur = defn[1]
            pc = 0
            stack = []
    return (None, "OM")

# Compile (with a cache for program lines) and evaluate the expression at toks[pos]. Returns (value, next position, error).
def eval_at(st, toks, pos):
    line = st["line"]
    cache_key = line * 4096 + pos
    hit = None
    if line > 0:
        hit = st["rpn"].get(cache_key)
    if hit == None:
        hit = compile_expr(toks, pos)
        if line > 0:
            st["rpn"][cache_key] = hit
    if hit[2] != "":
        return (None, hit[1], hit[2])
    r = eval_rpn(st, hit[0])
    return (r[0], hit[1], r[1])
```

lex.star

The tokenizer: keywords anywhere in a line, two-character names, numbers

```
# The tokenizer for TRS-80 Level II BASIC.
#
# It follows the real machine on purpose: keywords are recognised anywhere in a line, even in the middle of a name, so
# FORI=1TO10 works with no spaces and so does the famous trap that a variable called TOTAL is read as TO TAL. Names have
# two significant characters. Everything outside a string is case-insensitive.
#
# A token is a tuple whose first item says what it is:
#   ("n", value)              a number (int, single float, or a double held in a 1-tuple, see numbers.star)
#   ("s", text)               a string literal, or the comment text after REM, or the raw text of a DATA statement
#   ("v", name, suffix)       a variable name (two characters) and its type suffix: "" or one of $ % ! #
#   ("k", WORD)               a keyword
#   ("o", op)                 an operator or punctuation mark

load("numbers.star", "scan_number", "INT_MAX")

KEYWORDS = [
    # statements
    "END", "FOR", "NEXT", "DATA", "INPUT", "DIM", "READ", "LET", "GOTO", "RUN", "IF", "RESTORE", "GOSUB", "RETURN",
    "REM", "STOP", "ELSE", "TRON", "TROFF", "DEFSTR", "DEFINT", "DEFSNG", "DEFDBL", "ERROR", "RESUME", "ON", "POKE",
    "PRINT", "CONT", "LIST", "DELETE", "CLEAR", "NEW", "CLS", "SET", "RESET", "RANDOM", "DEF", "OUT", "EDIT",
    # words that sit inside statements
    "TAB(", "SPC(", "TO", "FN", "USING", "THEN", "NOT", "STEP", "AND", "OR",
    # functions
    "SGN", "INT", "ABS", "FRE", "INP", "POS", "SQR", "RND", "LOG", "EXP", "COS", "SIN", "TAN", "ATN", "PEEK", "CINT",
    "CSNG", "CDBL", "FIX", "LEN", "STR$", "VAL", "ASC", "CHR$", "LEFT$", "RIGHT$", "MID$", "INSTR", "STRING$",
    "POINT", "MEM", "INKEY$", "ERL", "ERR",
]

# Group the keywords by first letter, longest first, so keyword_at tries STRING$ before STR and INT before IN.
def _index_keywords():
    index = {}
    # Longest first, so that STRING$ is not read as STR and INT is not read as IN.
    for word in sorted(KEYWORDS, key = lambda w: -len(w)):
        index.setdefault(word[0], []).append(word)
    return index

KEYWORD_INDEX = _index_keywords()

# The keyword that starts at position i of the (upper-case) text, or "". Keywords are found anywhere, even in the middle of a name, as on Level II.
def keyword_at(u, i):
    for word in KEYWORD_INDEX.get(u[i], []):
        if u.startswith(word, i):
            return word
    return ""

# True for the characters "0" to "9".
def _is_digit(c):
    return c >= "0" and c <= "9"

# True for the upper-case letters "A" to "Z". The text is upper-cased before it is scanned.
def _is_letter(c):
    return c >= "A" and c <= "Z"

# The value of a hexadecimal digit (0-9, A-F), or -1.
def _hex_value(c):
    if c >= "0" and c <= "9":
        return ord_digit(c)
    if c >= "A" and c <= "F":
        return 10 + "ABCDEF".find(c)
```

```
    return -1

# The value of a decimal digit character, or -1.
def ord_digit(c):
    return "0123456789".find(c)

# The text of a DATA statement: everything up to the next colon that is not inside quotes.
def _data_end(t, i):
    n = len(t)
    in_quotes = False
    for k in range(i, n):
        c = t[k]
        if c == "\"":
            in_quotes = not in_quotes
        elif c == ":" and not in_quotes:
            return k
    return n

# Split a line of text into tokens. Returns (tokens, error code); the error code is "" when all is well.
def tokenize(text):
    t = text
    u = text.upper()
    if len(u) != len(t):
        u = t
    n = len(t)
    tokens = []
    i = 0
    for _ in range(n + 2):
        if i >= n:
            break
        c = u[i]
        if c == " ":
            i += 1
        elif c == "\"":
            j = t.find("\"", i + 1)
            if j < 0:
                tokens.append(("s", t[i + 1:]))
                i = n
            else:
                tokens.append(("s", t[i + 1:j]))
                i = j + 1
        elif _is_digit(c) or (c == "." and i + 1 < n and _is_digit(u[i + 1])):
            r = scan_number(u, i)
            if r[2] == -2:
                return (tokens, "OV")
            if r[2] == -1:
                return (tokens, "SN")
            tokens.append(("n", r[0]))
            i = r[1]
        elif c == "&":
            base = 0
            if i + 1 < n and u[i + 1] == "H":
                base = 16
            elif i + 1 < n and u[i + 1] == "O":
                base = 8
            if base == 0:
                return (tokens, "SN")
            j = i + 2
            value = 0
            for k in range(j, n):
                d = _hex_value(u[k])
                if d < 0 or d >= base:
                    break
                value = value * base + d
                j = k + 1
            if value > 65535:
                return (tokens, "OV")
            if value > INT_MAX:
                value -= 65536
            tokens.append(("n", value))
            i = j
        elif _is_letter(c):
            k = keyword_at(u, i)
            if k != "":
                i += len(k)
```

```
                if k == "REM":
                    tokens.append(("k", "REM"))
                    tokens.append(("s", t[i:]))
                    i = n
                elif k == "DATA":
                    end = _data_end(t, i)
                    tokens.append(("k", "DATA"))
                    tokens.append(("s", t[i:end]))
                    i = end
                else:
                    tokens.append(("k", k))
            else:
                j = i + 1
                for m in range(j, n):
                    d = u[m]
                    if not (_is_letter(d) or _is_digit(d)):
                        break
                    if keyword_at(u, m) != "":
                        break
                    j = m + 1
                name = u[i:j][:2]
                suffix = ""
                if j < n and (u[j] == "$" or u[j] == "%" or u[j] == "!" or u[j] == "#"):
                    suffix = u[j]
                    j += 1
                tokens.append(("v", name, suffix))
                i = j
        elif c == "'":
            tokens.append(("k", "REM"))
            tokens.append(("s", t[i + 1:]))
            i = n
        elif c == "?":
            tokens.append(("k", "PRINT"))
            i += 1
        elif c == "<" or c == ">" or c == "=":
            pair = u[i:i + 2]
            if pair == "<>" or pair == "><":
                tokens.append(("o", "<>"))
                i += 2
            elif pair == "<=" or pair == "=<":
                tokens.append(("o", "<="))
                i += 2
            elif pair == ">=" or pair == "=>":
                tokens.append(("o", ">="))
                i += 2
            else:
                tokens.append(("o", c))
                i += 1
        elif c in "+-*/^(),;:#@":
            tokens.append(("o", c))
            i += 1
        else:
            return (tokens, "SN")
    return (tokens, "")
```

numbers.star

Numbers: integer, single and double precision, printing, and the math functions

```
# Numbers for the BASIC interpreter.
#
# TRS-80 Level II BASIC has three numeric types: 16-bit integers, 32-bit single precision and 64-bit double precision.
# Here an integer is a Starlark int, a single is a float that has been rounded to 24 bits of mantissa (to_single), and a
# double is a one-element tuple holding a float, so the three can be told apart by type() alone.
#
# Starlark has no math module and no exceptions, so this file also holds the functions BASIC needs (SQR, EXP, LOG, SIN,
# COS, ATN, ^) written with series, and the number scanner that never fails on bad text.

INT_MIN = -32768
INT_MAX = 32767
SNG_LIMIT = 1.7014118346046923e38      # 2**127: a single at or above this overflows (?OV ERROR)
LN2 = 0.6931471805599453
PI = 3.141592653589793
HALF_PI = 1.5707963267948966
TWO_PI = 6.283185307179586

# Build the table of exact powers of two from 2**-170 to 2**170, indexed by exponent. Multiplying or halving by two is exact in floating point,
# so every entry is exact.
def _make_pow2():
    table = {}
    x = 1.0
    for k in range(0, 171):
        table[k] = x
        x = x * 2.0
    x = 1.0
    for k in range(0, 171):
        table[-k] = x
        x = x / 2.0
    return table

# Exact powers of two, 2**-170 .. 2**170.
POW2 = _make_pow2()

# ------------------------------------------------------------------ kinds

# 0 integer, 1 single, 2 double, 3 string.
def kind_of(v):
    t = type(v)
    if t == "int":
        return 0
    if t == "float":
        return 1
    if t == "tuple":
        return 2
    return 3

# Any BASIC number as a float. A double is stored as a one-item tuple (so it can be told from a single), and this unwraps it.
def to_float(v):
    if type(v) == "tuple":
        return v[0]
    return float(v)

# Largest integer not greater than x, as an int (floor), safe for any float.
def floor_int(x):
    if x != x:
        return 0
    n = int(x)
    if n > x:
        n -= 1
    return n

# The negative of a number, keeping its type. -32768 has no 16-bit negative, so it becomes the single-precision 32768.0.
def negate(v):
```

```
    k = kind_of(v)
    if k == 0:
        if v == INT_MIN:
            return 32768.0
        return -v
    if k == 1:
        return -v
    return (-v[0],)

# --------------------------------------------------------------------- single precision

# e such that 2**e <= a < 2**(e+1), for 2**-170 <= a < 2**170.
def binary_exponent(a):
    lo = -170
    hi = 170
    for _ in range(10):
        mid = (lo + hi + 1) // 2
        if POW2[mid] <= a:
            lo = mid
        else:
            hi = mid - 1
    return lo

# Round a float to the nearest 32-bit single (24-bit mantissa), ties to even. Values too small to
  represent become 0.
# The caller has already checked for overflow.
def to_single(x):
    if x == 0.0 or x != x:
        return 0.0
    a = x
    sign = 1.0
    if a < 0.0:
        a = -a
        sign = -1.0
    if a < POW2[-127]:
        return 0.0
    if a >= SNG_LIMIT:
        return sign * SNG_LIMIT
    e = binary_exponent(a)
    scale = POW2[23 - e]
    q = a * scale
    n = int(q)
    frac = q - n
    if frac > 0.5 or (frac == 0.5 and n % 2 == 1):
        n += 1
    return sign * (n / scale)

# True if a value is too big for single precision, or is not a number.
def overflows(x):
    return x >= SNG_LIMIT or x <= -SNG_LIMIT or x != x

# ------------------------------------------------------------------- printing numbers

# Significant digits of a > 0 as (digit string, decimal exponent of the first digit).
def _sci_digits(a, nd):
    s = ("%." + str(nd - 1) + "e") % a
    parts = s.split("e")
    return (parts[0].replace(".", ""), int(parts[1]))

# A float the way BASIC prints it, without the sign slot: nd significant digits, fixed notation for 0.01
  up to 10**nd,
# otherwise d.dddE+xx (D for double precision).
def format_real(x, nd, expc):
    if x == 0.0:
        return "0"
    sign = ""
    a = x
    if x < 0.0:
        sign = "-"
        a = -x
    d = _sci_digits(a, nd)
    digits = d[0].rstrip("0")
    e = d[1]
    if digits == "":
        digits = "0"
```

```
    if e >= -2 and e < nd:
        if e < 0:
            body = "." + "0" * (-e - 1) + digits
        elif len(digits) <= e + 1:
            body = digits + "0" * (e + 1 - len(digits))
        else:
            body = digits[:e + 1] + "." + digits[e + 1:]
    else:
        body = digits[0]
        if len(digits) > 1:
            body += "." + digits[1:]
        body += expc + ("-" if e < 0 else "+") + ("%02d" % abs(e))
    return sign + body

# A number as text with no sign slot (what STR$ builds on).
def number_text(v):
    k = kind_of(v)
    if k == 0:
        return str(v)
    if k == 1:
        return format_real(v, 6, "E")
    return format_real(v[0], 16, "D")

# True if a number is below zero, for any of the three numeric types.
def _is_negative(v):
    k = kind_of(v)
    if k == 2:
        return v[0] < 0.0
    return v < 0

# STR$: positive numbers get a leading space where the sign would be.
def str_text(v):
    s = number_text(v)
    if _is_negative(v):
        return s
    return " " + s

# PRINT: a leading space for the sign and a trailing space.
def print_text(v):
    return str_text(v) + " "

# ------------------------------------------------------------------- reading numbers

# The index just after the run of decimal digits that starts at s[i] (i itself if there are none).
def span_digits(s, i):
    j = i
    for k in range(i, len(s)):
        c = s[k]
        if c < "0" or c > "9":
            break
        j = k + 1
    return j

# Scan an unsigned number at s[i] (s already upper case). Returns (value, next index, kind):
# kind 0/1/2 is the type of the value, -1 means there is no number there, -2 means it overflows.
def scan_number(s, i):
    n = len(s)
    a = span_digits(s, i)
    j = a
    has_dot = False
    frac_end = a
    if j < n and s[j] == ".":
        has_dot = True
        frac_end = span_digits(s, j + 1)
        j = frac_end
    ndig = (a - i)
    if has_dot:
        ndig += frac_end - a - 1
    if ndig == 0:
        return (0, i, -1)
    # Only significant digits decide between single and double precision, so leading zeros do not count.
    sig_text = s[i:a]
    if has_dot:
        sig_text += s[a + 1:frac_end]
    sig = len(sig_text.lstrip("0"))
```

```
    mant_end = j
    has_exp = False
    dbl = False
    exp_sign = ""
    exp_digits = ""
    if j < n and (s[j] == "E" or s[j] == "D"):
        k = j + 1
        if k < n and (s[k] == "+" or s[k] == "-"):
            exp_sign = s[k]
            k += 1
        e_end = span_digits(s, k)
        if e_end > k:
            has_exp = True
            dbl = s[j] == "D"
            exp_digits = s[k:e_end]
            j = e_end
    text = s[i:mant_end]
    ev = 0
    if has_exp:
        if len(exp_digits) > 6:
            ev = 999999
        else:
            ev = int(exp_digits)
        if exp_sign == "-":
            ev = -ev
    # float() refuses text that is too large, so refuse it first. Leading zeros are not counted.
    int_digits = len(s[i:a].lstrip("0"))
    if int_digits + ev > 40:
        return (0, j, -2)
    value = 0.0
    if not (has_exp and ev < -400):
        value = float(text + ("e" + exp_sign + exp_digits if has_exp else ""))
    suffix = ""
    if j < n and (s[j] == "%" or s[j] == "!" or s[j] == "#"):
        suffix = s[j]
        j += 1
    if value >= SNG_LIMIT:
        return (0, j, -2)
    if suffix == "%":
        iv = floor_int(value)
        if iv > INT_MAX:
            return (0, j, -2)
        return (iv, j, 0)
    if suffix == "!":
        return (to_single(value), j, 1)
    if suffix == "#" or dbl or sig > 7:
        return ((value,), j, 2)
    if has_dot or has_exp:
        return (to_single(value), j, 1)
    iv = int(text)
    if iv <= INT_MAX:
        return (iv, j, 0)
    return (to_single(value), j, 1)

# VAL: the number at the start of s (after spaces), or 0.
def val_text(s):
    u = s.upper()
    n = len(u)
    i = 0
    for k in range(n):
        if u[k] != " ":
            break
        i = k + 1
    neg = False
    if i < n and u[i] == "-":
        neg = True
        i += 1
    elif i < n and u[i] == "+":
        i += 1
    r = scan_number(u, i)
    if r[2] < 0:
        return 0
    if neg:
        return negate(r[0])
    return r[0]
```

```
288
289 # A whole field of INPUT or DATA as a number: (ok, value). Empty is 0.
290 def parse_number_text(s):
291     u = s.upper().strip()
292     if u == "":
293         return (True, 0)
294     i = 0
295     neg = False
296     if u[0] == "-":
297         neg = True
298         i = 1
299     elif u[0] == "+":
300         i = 1
301     r = scan_number(u, i)
302     if r[2] < 0 or r[1] != len(u):
303         return (False, 0)
304     if neg:
305         return (True, negate(r[0]))
306     return (True, r[0])
307
308 # ------------------------------------------------------------------ math
309
310 # Square root of x >= 0 by Newton's method from a power-of-two guess.
311 def f_sqr(x):
312     if x <= 0.0:
313         return 0.0
314     g = POW2[binary_exponent(x) // 2]
315     for _ in range(8):
316         g = 0.5 * (g + x / g)
317     return g
318
319 # e**x, or None if it overflows a single.
320 def f_exp(x):
321     if x > 88.7:
322         return None
323     if x < -100.0:
324         return 0.0
325     if x >= 0.0:
326         k = int(x / LN2 + 0.5)
327     else:
328         k = int(x / LN2 - 0.5)
329     r = x - k * LN2
330     term = 1.0
331     s = 1.0
332     for i in range(1, 17):
333         term = term * r / i
334         s += term
335     return s * POW2[k]
336
337 # Natural logarithm of x > 0.
338 def f_log(x):
339     e = binary_exponent(x)
340     m = x / POW2[e]
341     if m > 1.4142135623730951:
342         m = m / 2.0
343         e += 1
344     z = (m - 1.0) / (m + 1.0)
345     z2 = z * z
346     t = z
347     s = z
348     for i in range(1, 14):
349         t = t * z2
350         s += t / (2 * i + 1)
351     return e * LN2 + 2.0 * s
352
353 # Sine by reduction and series: bring x into -pi/2..pi/2 using the periodicity and symmetry of sine, then
      sum the Taylor series. Good to the six
354 # or seven digits Level II printed. (The separate package std/approx_math.star is a more accurate, general
      version.)
355 def f_sin(x):
356     if x >= 0.0:
357         k = int(x / TWO_PI + 0.5)
358     else:
359         k = int(x / TWO_PI - 0.5)
360     r = x - k * TWO_PI
```

```
    if r > HALF_PI:
        r = PI - r
    elif r < -HALF_PI:
        r = -PI - r
    r2 = r * r
    term = r
    s = r
    for i in range(1, 13):
        term = -term * r2 / ((2 * i) * (2 * i + 1))
        s += term
    return s

# Cosine, as the sine of x plus a quarter turn.
def f_cos(x):
    return f_sin(x + HALF_PI)

# Arc tangent: fold x into 0..1 using atan(-x) = -atan(x) and atan(1/x) = pi/2 - atan(x), halve the angle twice to make the series converge
# fast, sum the series, and undo the folding.
def f_atn(x):
    neg = x < 0.0
    a = x
    if neg:
        a = -x
    inv = a > 1.0
    if inv:
        a = 1.0 / a
    for _ in range(2):
        a = a / (1.0 + f_sqr(1.0 + a * a))
    a2 = a * a
    term = a
    s = a
    for i in range(1, 14):
        term = -term * a2
        s += term / (2 * i + 1)
    s = s * 4.0
    if inv:
        s = HALF_PI - s
    if neg:
        return -s
    return s

# x ** y as (value, error code). Error codes are the BASIC ones: "" none, "FC", "OV", "/0".
def f_pow(x, y):
    if y == 0.0:
        return (1.0, "")
    if x == 0.0:
        if y > 0.0:
            return (0.0, "")
        return (0.0, "/0")
    if y == float(floor_int(y)) and abs(y) <= 2048.0:
        n = floor_int(abs(y))
        result = 1.0
        base = x
        for _ in range(12):
            if n % 2 == 1:
                result = result * base
            n = n // 2
            if n == 0:
                break
            base = base * base
        if y < 0.0:
            if result == 0.0 or overflows(result):
                return (0.0, "OV")
            result = 1.0 / result
        if overflows(result):
            return (0.0, "OV")
        return (result, "")
    if x < 0.0:
        return (0.0, "FC")
    t = y * f_log(x)
    r = f_exp(t)
    if r == None:
        return (0.0, "OV")
    return (r, "")
```

```
435
436 # Next state of the pseudo random generator (a 31-bit linear congruential generator) and its value in [0,
      1).
437 def rnd_next(seed):
438     return (seed * 1103515245 + 12345) % 2147483648
```

screen.star

The 64 x 16 screen, block graphics and PRINT USING

```
# The TRS-80 screen: 16 rows of 64 characters, and 128 x 48 block graphics.
#
# The screen is a list of 1024 character codes, the same as the video memory the real machine had at addresses 15360 to
# 16383, so PRINT @, POKE, PEEK, SET, RESET and POINT all work on one store. Codes 32 to 127 are text, and 128 to 191 are
# graphics cells of 2 x 3 blocks (code 128 plus one bit per block).
#
# Everything printed is also appended to a plain text stream (st["out"]) so a terminal host can show it as it comes, and
# the tests can read it without caring about the screen.

load("numbers.star", "floor_int", "to_float", "kind_of", "overflows")

COLUMNS = 64
ROWS = 16
CELLS = 1024
VIDEO_BASE = 15360

# Character -> screen code, for printing a character into video memory. Characters outside 0 to 255 are stored as 63, "?".
CODES = {"%c" % i: i for i in range(256)}

# Build the table that shows the 64 possible 2 x 3 block-graphics cells as Unicode "sextant" characters (and the half and full blocks, which
# Unicode keeps elsewhere). Index is the six-bit pattern, one bit per block.
def _make_sextants():
    table = []
    for pattern in range(64):
        if pattern == 0:
            table.append(" ")
        elif pattern == 21:
            table.append("%c" % 0x258C)
        elif pattern == 42:
            table.append("%c" % 0x2590)
        elif pattern == 63:
            table.append("%c" % 0x2588)
        else:
            skip = 0
            if pattern > 21:
                skip += 1
            if pattern > 42:
                skip += 1
            table.append("%c" % (0x1FB00 + pattern - 1 - skip))
    return table

SEXTANTS = _make_sextants()

# A blank screen: 1,024 cells (16 rows of 64) all holding the space character (32).
def new_screen():
    return [32] * CELLS

# Move every row up one and blank the bottom row; the cursor goes to the start of the bottom row.
def _scroll(st):
    st["screen"] = st["screen"][COLUMNS:] + [32] * COLUMNS
    st["cur"] = CELLS - COLUMNS

# Move the cursor to the start of the next row, scrolling if that was the last one.
def newline(st):
    cur = (st["cur"] // COLUMNS + 1) * COLUMNS
    st["cur"] = cur
    if cur >= CELLS:
        _scroll(st)
    st["col"] = st["cur"] % COLUMNS

# Write text at the cursor. Newlines move to the start of the next line, and a line that reaches column 64 wraps.
def put_text(st, s):
    if not st["quiet"]:
        st["out"].append(s)
```

```
    screen = st["screen"]
    for ch in s.elems():
        if ch == "\n" or ch == "\r":
            newline(st)
            screen = st["screen"]
            continue
        screen[st["cur"]] = CODES.get(ch, 63)
        st["cur"] += 1
        if st["cur"] >= CELLS:
            _scroll(st)
            screen = st["screen"]
        st["col"] = st["cur"] % COLUMNS

# CLS: blank the screen, home the cursor, and put a form feed in the output stream so a terminal host can clear too.
def clear_screen(st):
    st["screen"] = new_screen()
    st["cur"] = 0
    st["col"] = 0
    st["out"].append("\f")

# Move the cursor to the cell `pos` (0 to 1023), as PRINT @ does.
def set_cursor(st, pos):
    st["cur"] = pos
    st["col"] = pos % COLUMNS

# ------------------------------------------------------------------ graphics

# Which character cell holds the block-graphics dot (x, y), and which of the cell's six bits it is. A cell is 2 dots wide and 3 high.
def _cell_and_bit(x, y):
    return ((y // 3) * COLUMNS + x // 2, 1 << ((y % 3) * 2 + x % 2))

# Turn the dot (x, y) on or off. A cell that held text becomes a graphics cell (codes 128 to 191); one whose dots are all off stays a blank
# graphics cell (128), not the space character.
def set_pixel(st, x, y, on):
    c = _cell_and_bit(x, y)
    code = st["screen"][c[0]]
    if code < 128 or code > 191:
        code = 128
    if on:
        code = code | c[1]
    else:
        code = code & (255 - c[1])
        if code < 128:
            code = 128
    st["screen"][c[0]] = code

# True if the dot (x, y) is on: the cell must be a graphics cell and the dot's bit set. This is what POINT reads.
def point_on(st, x, y):
    c = _cell_and_bit(x, y)
    code = st["screen"][c[0]]
    return code >= 128 and code <= 191 and (code & c[1]) != 0

# ------------------------------------------------------------------ showing the screen

# The character to show for a screen code: text for 32 to 127, a sextant for 128 to 191, a space for anything else.
def _glyph(code):
    if code >= 32 and code < 128:
        return "%c" % code
    if code >= 128 and code < 192:
        return SEXTANTS[code - 128]
    return " "

# The 16 lines of the screen as text, with trailing spaces removed.
def screen_lines(st):
    screen = st["screen"]
    lines = []
    for r in range(ROWS):
        row = screen[r * COLUMNS:(r + 1) * COLUMNS]
        lines.append("".join([_glyph(c) for c in row]).rstrip(" "))
    return lines
```

```
135
136 # ---------- PRINT USING
137
138 # How many times the character `ch` repeats in s starting at i. Used to measure runs of # in a PRINT USING
      template.
139 def _count(s, i, ch):
140     j = i
141     for k in range(i, len(s)):
142         if s[k] != ch:
143             break
144         j = k + 1
145     return j - i
146
147 # Cut a format string into literal text, numeric fields and string fields.
148 # A numeric field is ("num", dict), a string field is ("str", width) with width 0 meaning "the whole
      string" and -1 the
149 # first character only.
150 def parse_using(fmt):
151     segs = []
152     lit = ""
153     n = len(fmt)
154     i = 0
155     for _ in range(n + 1):
156         if i >= n:
157             break
158         c = fmt[i]
159         if c == "_" and i + 1 < n:
160             lit += fmt[i + 1]
161             i += 2
162         elif c == "!":
163             if lit != "":
164                 segs.append(("lit", lit))
165                 lit = ""
166             segs.append(("str", -1))
167             i += 1
168         elif c == "&":
169             if lit != "":
170                 segs.append(("lit", lit))
171                 lit = ""
172             segs.append(("str", 0))
173             i += 1
174         elif c == "\\":
175             j = fmt.find("\\", i + 1)
176             if j < 0:
177                 lit += c
178                 i += 1
179             else:
180                 if lit != "":
181                     segs.append(("lit", lit))
182                     lit = ""
183                 segs.append(("str", j - i + 1))
184                 i = j + 1
185         elif c == "#" or c == "." or c == "+" or c == "*" or c == "$" or (c == "-" and False):
186             spec = {"lead_plus": False, "stars": False, "dollar": False, "int": 0, "frac": 0, "dot": False,
187                     "commas": False, "trail": "", "exp": False, "width": 0}
188             j = i
189             if fmt[j] == "+":
190                 spec["lead_plus"] = True
191                 j += 1
192             if fmt.startswith("**", j):
193                 spec["stars"] = True
194                 spec["int"] += 2
195                 j += 2
196             if fmt.startswith("$$", j):
197                 spec["dollar"] = True
198                 spec["int"] += 1
199                 j += 2
200             for _ in range(n + 1):
201                 if j >= n:
202                     break
203                 d = fmt[j]
204                 if d == "#":
205                     spec["int"] += 1
206                     j += 1
207                 elif d == "," and not spec["dot"]:
```

```
                    spec["commas"] = True
                    j += 1
                else:
                    break
            if j < n and fmt[j] == "." and (j + 1 < n and fmt[j + 1] == "#" or spec["int"] > 0):
                spec["dot"] = True
                j += 1
                k = _count(fmt, j, "#")
                spec["frac"] = k
                j += k
            if fmt.startswith("^^^^", j):
                spec["exp"] = True
                j += 4
            if j < n and (fmt[j] == "-" or fmt[j] == "+") and not spec["lead_plus"]:
                spec["trail"] = fmt[j]
                j += 1
            if spec["int"] + spec["frac"] == 0:
                lit += fmt[i:j] if j > i else c
                i = max(j, i + 1)
            else:
                if lit != "":
                    segs.append(("lit", lit))
                    lit = ""
                spec["width"] = j - i
                segs.append(("num", spec))
                i = j
        else:
            lit += c
            i += 1
    if lit != "":
        segs.append(("lit", lit))
    return segs

# Put a comma between every group of three digits, from the right: "1234567" becomes "1,234,567".
def _with_commas(digits):
    out = ""
    n = len(digits)
    for k in range(n):
        if k > 0 and (n - k) % 3 == 0:
            out += ","
        out += digits[k]
    return out

# Format x in a numeric field.
def format_field(x, spec):
    neg = x < 0.0
    a = -x if neg else x
    if spec["exp"]:
        body = ("%." + str(spec["frac"] + spec["int"] - 1 if spec["int"] > 0 else spec["frac"]) + "e") % a
        parts = body.split("e")
        mant = parts[0]
        ev = int(parts[1])
        text = mant + "E" + ("-" if ev < 0 else "+") + ("%02d" % abs(ev))
    else:
        text = ("%." + str(spec["frac"]) + "f") % a
        if spec["commas"]:
            ip = text.split(".")
            text = _with_commas(ip[0]) + (("." + ip[1]) if len(ip) > 1 else "")
        if spec["dot"] and spec["frac"] == 0:
            text += "."
    if spec["dollar"]:
        text = "$" + text
    sign = ""
    if neg:
        sign = "-"
    elif spec["lead_plus"]:
        sign = "+"
    if spec["trail"] != "":
        tail = ""
        if neg:
            tail = "-"
        elif spec["trail"] == "+":
            tail = "+"
        else:
            tail = " "
```

```
        full = text + tail
    else:
        full = sign + text
    if len(full) > spec["width"]:
        return "%" + full
    fill = "*" if spec["stars"] else " "
    return fill * (spec["width"] - len(full)) + full

# Format values by a PRINT USING format string. Returns (text, error).
def format_using(fmt, values):
    segs = parse_using(fmt)
    fields = 0
    for s in segs:
        if s[0] != "lit":
            fields += 1
    if fields == 0:
        return ("", "FC")
    out = ""
    vi = 0
    n = len(values)
    for _ in range(n * len(segs) + len(segs) + 2):
        done = False
        for s in segs:
            if s[0] == "lit":
                out += s[1]
            else:
                if vi >= n:
                    done = True
                    break
                v = values[vi]
                vi += 1
                if s[0] == "num":
                    if kind_of(v) == 3:
                        return ("", "TM")
                    out += format_field(to_float(v), s[1])
                else:
                    if kind_of(v) != 3:
                        return ("", "TM")
                    w = s[1]
                    if w == 0:
                        out += v
                    elif w == -1:
                        out += v[:1] if v != "" else " "
                    else:
                        out += (v + " " * w)[:w]
        if done or vi >= n:
            break
    return (out, "")
```

editor.star

The EDIT command as a function of one keystroke

```
# The Level II EDIT command, as a function of one keystroke.
#
# EDIT is not line oriented. After `EDIT 100` the machine shows the line number and then reacts to every key at once: a space steps
# the cursor over the next character (which makes it appear), D deletes, I inserts, and so on. This file holds that editor as a
# pure state machine so it can be tested on its own: edit_step(e, key) changes the dict `e` and returns (text to show, action),
# where action is "" (carry on), "save" (store the edited line and finish) or "quit" (finish and keep the original).
#
# The line is held as a list of characters (code points), so a character outside the basic plane is one step, never half of one.
# A key is a one character string, or one of:  "\r" Enter,  "\x1b" Escape,  "\b" Backspace.
#
# Subcommands (n is an optional count typed first, 1 if absent):
#   Space        n   move right over n characters, showing them
#   Backspace    n   move left n characters (in insert mode: delete the character just typed)
#   L                list the rest of the line, then go back to the start of it
#   D            n   delete n characters; the deleted text is shown between backslashes
#   C            n   change the next n characters: each key typed replaces one
#   I                insert what is typed at the cursor, until Escape
#   X                go to the end of the line and insert there
#   H                hack: delete the rest of the line and insert there
#   S c          n   search: move to just before the nth c after the cursor, showing what is passed (the end if there is none)
#   K c          n   kill: delete up to just before the nth c after the cursor (nothing if there is none)
#   Q                quit: leave the line as it was
#   A                abort: throw away the changes so far and start the line again
#   E                end: store the changes and leave, without showing the rest of the line
#   Enter            store the changes, show the rest of the line and leave
#   Escape           leave insert or change mode, or cancel a count or a pending S or K
#
# Not checked against a real machine (written from the manual as remembered): the backslash around deleted text, that S starts at the
# character after the cursor, that a failed S runs to the end, and that a failed K does nothing. Each is one line below.

MAX_LENGTH = 255
DELETE_MARK = "\\"

# The text shown at the start of an edit and after L or A: the line number and a space.
def edit_head(e):
    return "%d " % e["line"]

# A new edit of line `line` whose text (after the line number) is `text`.
def new_edit(line, text):
    chars = list(text.codepoints())
    return {
        "line": line, "orig": list(chars), "buf": chars, "pos": 0,
        "mode": "cmd",        # "cmd", "insert" or "change"
        "count": 0,           # digits typed so far
        "pending": "",        # "S" or "K" while waiting for the character to look for
        "arg_count": 1,       # the count that was typed before that S or K
        "left": 0,            # characters still to change in change mode
        "insert_from": 0,     # where this insert began; backspace never goes before it
    }

# The line as it is now, as a string.
def edit_text(e):
    return "".join(e["buf"])

# True for a one-character key that can go into the line (not a control key or DEL).
def _printable(key):
    return len(key) > 0 and ord(key) >= 32 and ord(key) != 127

# True for the keys that mean backspace: BS (8) and DEL (127).
def _is_backspace(key):
    return key == "\b" or key == "\x7f"

```

```
# Feed the editor one key. Returns (text to show, action) where action is "" (carry on), "save" or "quit". Enter always saves. Otherwise the key goes
# to the handler for the current mode: inserting, changing, waiting for the character of S or K, or an ordinary command.
def edit_step(e, key):
    if key == "\r" or key == "\n":
        return ("".join(e["buf"][e["pos"]:]), "save")
    mode = e["mode"]
    if mode == "insert":
        return _insert_key(e, key)
    if mode == "change":
        return _change_key(e, key)
    if e["pending"] != "":
        return _argument_key(e, key)
    return _command_key(e, key)

# A key while inserting. Escape ends the insert. Backspace takes back the last typed character but never goes before where the insert began.
# Any printable key is put in at the cursor, up to the 255-character limit.
def _insert_key(e, key):
    if key == "\x1b":
        e["mode"] = "cmd"
        return ("", "")
    if _is_backspace(key):
        if e["pos"] > e["insert_from"]:
            e["pos"] -= 1
            e["buf"] = e["buf"][:e["pos"]] + e["buf"][e["pos"] + 1:]
            return ("\b \b", "")
        return ("", "")
    if _printable(key) and len(e["buf"]) < MAX_LENGTH:
        e["buf"] = e["buf"][:e["pos"]] + [key] + e["buf"][e["pos"]:]
        e["pos"] += 1
        return (key, "")
    return ("", "")

# A key in change mode (the C command): each printable key replaces the character under the cursor, until the counted number have been
# changed or Escape is pressed.
def _change_key(e, key):
    if key == "\x1b":
        e["mode"] = "cmd"
        e["left"] = 0
        return ("", "")
    if _printable(key) and e["pos"] < len(e["buf"]):
        e["buf"] = e["buf"][:e["pos"]] + [key] + e["buf"][e["pos"] + 1:]
        e["pos"] += 1
        e["left"] -= 1
        if e["left"] <= 0 or e["pos"] >= len(e["buf"]):
            e["mode"] = "cmd"
        return (key, "")
    return ("", "")

# The character that S (search) or K (kill) was waiting for. Finds the nth following occurrence, counting from the character after the cursor.
# S moves the cursor to just before it and shows what it passes (to the end of the line if there is none); K deletes up to just before it
# (and does nothing if there is none).
def _argument_key(e, key):
    command = e["pending"]
    e["pending"] = ""
    if not _printable(key):
        return ("", "")          # Escape (or any control key) cancels the S or K
    want = key.upper()
    buf = e["buf"]
    pos = e["pos"]
    found = -1
    seen = 0
    for j in range(pos + 1, len(buf)):
        if buf[j].upper() == want:
            seen += 1
            if seen == e["arg_count"]:
                found = j
                break
    if command == "S":
        if found < 0:
```

```
            found = len(buf)
        e["pos"] = found
        return ("".join(buf[pos:found]), "")
    if found < 0:
        return ("", "")
    e["buf"] = buf[:pos] + buf[found:]
    return (DELETE_MARK + "".join(buf[pos:found]) + DELETE_MARK, "")

# A key in command mode: a digit builds up a repeat count, then Space, Backspace, L, D, C, I, X, H, S, K,
  Q, A, E act as described at the top of this
# file. Any other key does nothing.
def _command_key(e, key):
    if len(key) == 1 and key >= "0" and key <= "9":
        e["count"] = min(255, e["count"] * 10 + int(key))
        return ("", "")
    n = e["count"] if e["count"] > 0 else 1
    e["count"] = 0
    buf = e["buf"]
    pos = e["pos"]
    if key == "\x1b":
        return ("", "")
    if key == " ":
        stop = min(len(buf), pos + n)
        e["pos"] = stop
        return ("".join(buf[pos:stop]), "")
    if _is_backspace(key):
        back = min(n, pos)
        e["pos"] = pos - back
        return ("\b" * back, "")
    k = key.upper()
    if k == "L":
        e["pos"] = 0
        return ("".join(buf[pos:]) + "\n" + edit_head(e), "")
    if k == "D":
        stop = min(len(buf), pos + n)
        if stop == pos:
            return ("", "")
        e["buf"] = buf[:pos] + buf[stop:]
        return (DELETE_MARK + "".join(buf[pos:stop]) + DELETE_MARK, "")
    if k == "C":
        if pos >= len(buf):
            return ("", "")
        e["mode"] = "change"
        e["left"] = n
        return ("", "")
    if k == "I":
        e["mode"] = "insert"
        e["insert_from"] = pos
        return ("", "")
    if k == "X":
        e["pos"] = len(buf)
        e["mode"] = "insert"
        e["insert_from"] = len(buf)
        return ("".join(buf[pos:]), "")
    if k == "H":
        e["buf"] = buf[:pos]
        e["mode"] = "insert"
        e["insert_from"] = pos
        return ("", "")
    if k == "S" or k == "K":
        e["pending"] = k
        e["arg_count"] = n
        return ("", "")
    if k == "Q":
        return ("", "quit")
    if k == "E":
        return ("", "save")
    if k == "A":
        e["buf"] = list(e["orig"])
        e["pos"] = 0
        e["mode"] = "cmd"
        e["pending"] = ""
        return ("\n" + edit_head(e), "")
    return ("", "")              # any other key does nothing
```